

--	--

S.E.P – Sistema E-commerce Panini

Documento de Arquitetura de Software

Versão 2.0

--	--

Histórico de Revisões			
Versão	Data	Descrição	Autor
1.0	14/05/2026	Criação inicial do Documento e definição da estrutura base do projeto.	Douglas Masuzzo Brasílio
1.1	21/05/2026	Elaboração da seção de Introdução e subseções introdutórias.	Douglas Masuzzo Brasílio e Martha Beatriz Valério
1.2	21/05/2026	Desenvolvimento da Representação Arquitetural do sistema, definição do modelo arquitetural adotado e visões arquitetônicas do sistema.	Gabriel Krieger
1.3	28/05/2026	Especificação de Metas e Restrições da Arquitetura, contemplando levantamento de requisitos, limitações técnicas e diretrizes arquitetônicas.	Gabriel Krieger
1.4	04/06/2026	Construção da Visão de Caso de Uso, e descrição dos principais cenários de interação entre atores.	David do Carmo
1.5	04/06/2026	Desenvolvimento da Visão Lógica, incluindo a organização estrutural, pacotes arquitetônicos e definição entre camadas.	David do Carmo
1.6	05/06/2026	Elaboração da Visão de Processos, incluindo fluxos operacionais, mecanismo de comunicação entre componentes e processos lógicos.	Martha Beatriz Valério
1.7	05/06/2026	Desenvolvimento da Visão de Implantação, descrevendo a infraestrutura, nós físicos e ambiente de hospedagem da solução.	Kayk Stefano Eduardo
1.8	06/06/2026	Elaboração da Visão de Implementação, incluindo componentes arquiteturais, tecnologias empregadas e organização dos subsistemas.	Kayk Stefano Eduardo
1.9	06/06/2026	Complementação da Visão de Implementação com detalhamento das camadas, fronteiras arquiteturais e visão geral de componentes.	Douglas Masuzzo Brasílio
2.0	07/06/2026	Revisão geral do documento, validação de consistência entre seções, correções textuais e consolidação da versão final.	Martha Beatriz Valério

Sumário

1. Introdução	4
---------------------	---

--	--

1.1. Escopo.....	4
1.2. Definições, Acrônimos e Abreviações	5
1.3. Público-Alvo	6
1.4. Visão Geral.....	7
2. Identificação do Projeto.....	7
3. Representação Arquitetural	8
4. Metas e Restrições da Arquitetura.....	9
4.1. Requisitos Funcionais	9
4.2. Requisitos Não-Funcionais	11
5. Visão de Casos de Uso.....	13
5.1. Realizações de Caso de Uso.....	13
1.1. Diagrama Geral de Caso de Uso	15
1.2. Realizações Individuais de Caso de Uso.....	16
UC01 – Cadastro de Usuários	16
UC04 – Gerenciamento de Catálogo	18
UC07 – Compra Online	18
UC11 – Gerenciamento de Pedidos	19
UC20 - Relatório de Vendas	20
6. Visão Lógica.....	21
6.1. Visão Geral.....	21
6.2. Pacotes de Design da Arquitetura	22
Pacote: dominio.usuario.....	23
Pacote: dominio.catalogo.....	24
Pacote: dominio.pedido.....	25
Pacote: dominio.pagamento	26
Pacote: dominio.vendas	27
6.3. Camadas	28
Camada de Apresentação (View)	28
Camada de Serviços	28
Camada de Persistência	29
Camada de Armazenamento e Integrações Externas	30
7. Visão de Processos	30
Processo 1: Realizar um pedido	31
PROCESSO 2: Pagamento e Checkout	32
8. Visão de Implantação.....	34
8.1. Descrição de Nós-Físicos	35
8.2. Mapeamento dos Processos de Nós-Físicos	37
9. Visão de Implementação.....	37
9.1. Visão Geral.....	38
9.2. Fronteiras de Camadas e Fluxos de Comunicações.....	40
9.3. Lista de Ferramentas por Camadas	40

--	--

10. Visão de Dados	42
11. Referências	45

--	--

Documento de Arquitetura de Software

1. INTRODUÇÃO

Este documento de Arquitetura de Software apresenta uma visão abrangente da Arquitetura do Sistema E-commerce Panini (S.E.P), desenvolvido no contexto da disciplina de Engenharia de Software III. O seu objetivo é documentar, organizar e comunicar as principais decisões arquitetônicas adotadas durante a concepção e o desenvolvimento do sistema, servindo como referência para a equipe de desenvolvimento, docentes avaliadores e demais partes interessadas no projeto.

O S.E.P consiste em uma plataforma digital de comércio eletrônico voltada à comercialização de produtos colecionáveis da Panini Brasil, oferecendo funcionalidades relacionadas ao gerenciamento de usuários, catálogos de produtos, promoções, pedidos, pagamentos, entregas e suporte ao cliente. A arquitetura do sistema é descrita utilizando o modelo de Visão Arquitetural 4+1, desenvolvido por Philippe Kruchten, permitindo representar diferentes perspectivas do software por meio de Visões de Casos de Uso, Lógica Processo, Implantação e Implementação.

Além disso, o projeto adota princípios modernos de arquitetura de software, incluindo a segregação de responsabilidades entre operações de leitura e escrita por meio do padrão CQRS, visando favorecer escalabilidade, organização e manutenção de solução. Este documento tem como finalidade de fornecer uma base técnica para implementação, validação e evolução do sistema, assegurando alinhamento entre os requisitos técnicos e decisões arquitetônicas.

1.1. Escopo

Este documento de Arquitetura de Software aplica-se ao Sistema E-commerce Panini (S.E.P), abrangendo a definição da arquitetura necessária para suporte as funcionalidades do sistema. O escopo arquitetural contempla:

- Portal web responsivo e compatível com dispositivos móveis.
- Cadastro, autenticação e gerenciamento de usuários.
- Gerenciamento de catálogo de produtos e promoções.
- Processo de compra online e gerenciamento de carrinho.
- Integração com gateways de pagamento.
- Controle e acompanhamento de pedidos.
- Rastreamento de entrega.
- Gestão de notificações e suporte ao cliente.

--	--

- Histórico de pedidos e relatórios gerenciais.
- Integrações com serviços externos de pagamento, autenticação e comunicação.
- Infraestrutura baseada em computação em nuvem, contêineres Docker e serviços de orquestração.

A arquitetura descrita neste documento cobre os requisitos funcionais e não funcionais definidos na Matriz de Rastreabilidade do projeto.

1.2. Definições, Acrônimos e Abreviações

A tabela a seguir representa todos os acrônimos, abreviações e definições de recursos e tecnologias utilizados neste documento e no desenvolvimento do sistema S.E.P.

Sigla / Acrônimo	Definição
API	<i>Application Programming Interface</i> (Interface de Comunicação de Sistemas)
AWS	<i>Amazon Web Services</i> (Provedores em nuvem pública da Amazon)
CDN	<i>Content Delivery Network</i> (Rede de distribuição de conteúdo estático)
CI/CD	<i>Continuous Integration / Continuous Delivery</i> (Automação de integração e entrega de software)
CMMI	<i>Capability Maturity Model Integration</i> (Modelo de maturidade de processos de desenvolvimento)
CRM	<i>Customer Relationship Management</i> (Sistema de gerenciamento de relacionamento com clientes)
CQRS	<i>Command Query Responsibility Segregation</i> (Separação de responsabilidades entre leitura e escrita)
Docker	Plataforma distribuída de streaming de eventos para comunicação assíncrona
Draw.io	Ferramenta gratuita de diagramação online integrada ao ecossistema Google
ERP	<i>Enterprise Resource Planning</i> (Sistema integrado de gestão empresarial)
GCP	<i>Google Cloud Platform</i> (Provedor de nuvem pública do Google)
HTTP/HTTPS	<i>Hypertext Transfer Protocol (Secure)</i> (Protocolo de comunicação web com criptografia TLS)
IEEE	<i>Institute of Electrical and Electronics Engineers</i> (Normas de Engenharia de Software)
ISO/IEC	<i>International Organization for Standardization</i> (Normas internacionais de qualidade)
ITIL	<i>Information Technology Infrastructure Library</i> (Boas práticas para gestão de serviços)
JWT	<i>JSON Web Token</i> (Mecanismo de autenticação baseado em tokens)

--	--

Kafka	Plataforma distribuída de streaming para eventos de comunicação assíncrona
Kubernetes	Orquestrador de contêineres para implantação escalável de microsserviços
LGPD	Lei Geral de Proteção de Dados (Lei nº 13.709/2018)
LucidChart	Ferramenta online de diagramação e visualização de fluxos e arquiteturas
MFA/2FA	<i>Multi-Factor Authentication</i> (Autenticação multifator para acesso)
MVC	<i>Model View Controller</i> (Padrão arquitetural de separação de responsabilidades)
OpenPix	Plataforma de pagamento instantâneo via Pix integrada ao sistema
Oracle	Empresa fornecedora de Banco de Dados Relacional corporativo e soluções de Middleware
ORM	<i>Object-Relational Mapping</i> (Mapeamento de Objeto Relacional)
PagBank	Gateway de pagamento integrado ao sistema para processar transações financeiras
REST	<i>Representational State Transfer</i> (Estilo arquitetural para API's web)
RTO	<i>Recovery Time Objective</i> (Tempo máximo de recuperação após falha)
SQL	<i>Structured Query Language</i> (Linguagem de consulta para banco de dados relacionais)
StarUML	Ferramenta (CASE) para modelagem de diagramas UML
TLS	<i>Transport Layer Security</i> (Protocolo criptográfico para comunicação segura)
UML	<i>Unified Model Language</i> (Linguagem de Modelagem de Sistemas de Softwares)
W3C	<i>World Wide Web Consortium</i> (Padrões e diretrizes para a web)

Tabela 1.2

1.3. Público-Alvo

Este documento de Arquitetura de Software destina-se aos diversos participantes envolvidos no desenvolvimento e avaliação do Sistema E-commerce Panini (S.E.P). Os principais públicos-alvo são:

- Equipe de Desenvolvimento: responsável pela implementação dos componentes de software e pela utilização das diretrizes arquitetônicas aqui definidas durante a construção do sistema.
- **Analistas e Arquitetos de Software:** responsáveis pela validação das decisões arquitetônicas, manutenção de consistências técnicas do projeto e evolução da solução ao longo do seu ciclo de vida.

--	--

- **Gerentes de Projeto:** utilizam-se deste documento como apoio para acompanhamento das atividades técnicas, gerenciamento de riscos e tomadas de decisões relacionadas ao desenvolvimento.
- **Stakeholders:** utilizam este documento para compreender a organização geral da solução, seus componentes principais e as decisões arquitetônicas adotadas.

1.4. Visão Geral

Este documento será organizado em seções apresentam diferentes perspectivas da arquitetura do S.E.P, seguindo o modelo arquitetural 4+1 de Philippe Kruchten.

- **Identificação do Projeto:** apresenta as informações gerais do sistema e dos responsáveis pelo desenvolvimento.
- **Representação Arquitetural:** descreve a abordagem arquitetônica adotada e as principais tecnologias utilizadas.
- **Metas e Restrições:** apresenta os requisitos não funcionais, limitações e objetivos que influenciam as decisões arquitetônicas.
- **Visão de Casos de Uso:** descreve os principais cenários de interação dos usuários com o sistema.
- **Visão Lógica:** apresenta a organização estrutural dos componentes de software, pacotes e camadas arquitetônicas.
- **Visão de Processos:** descreve os processos de negócios, fluxos de execuções e mecanismos de comunicação entre os componentes do sistema.
- **Visão de Implantação:** apresenta a infraestrutura física e lógica necessária para a execução da solução.
- **Visão de Implementação:** descreve a organização dos componentes de software, subsistemas e tecnologias empregadas.
- **Visão de Dados:** apresenta os aspectos relacionados ao armazenamento persistente de informações do sistema, incluindo entidades, relacionamentos e estratégias de persistência adotadas.
- **Tamanho e Desempenho:** apresenta considerações relacionadas à capacidade operacional do sistema, desempenho esperado, volume de dados processados e requisitos de escalabilidade em decisões arquitetônicas.
- **Qualidade:** apresenta os atributos de qualidade consideráveis ao sistema, descrevendo características relacionadas à segurança, confiabilidade, disponibilidade, manutenibilidade e demais funcionalidades orientadas a arquitetura.
- **Anexos:** contempla os materiais e conteúdos aplicados durante o desenvolvimento do projeto.

As seções finais apresentam informações complementares relacionadas à persistência de dados, desempenho, atributos de qualidade, anexos e referências bibliográficas.

--	--

2. Identificação do Projeto

Projeto	S.E.P - Sistema E-commerce Panini
Requisitante	Panini Brasil
Gerente de Projetos	Douglas Masuzzo Brasílio e Martha Beatriz Valério

3. Representação Arquitetural

Figura 1. Representação Arquitetural



Figura 3.1

Essa seção irá apresentar e detalhar as visões baseadas no modelo 4+1 (KRU41), utilizando os modelos definidos pelo Modelo de Documentação de Software (MDS). As visões utilizadas no documento são:

- **Visão de Casos de Uso:** Define os principais atores e suas interações com o sistema, oferecendo uma visão funcional de alto nível, alinhando com a experiência do usuário.
- **Visão Lógica:** Representa os componentes lógicos, como pacotes, classes, camadas e sistemas externos.
- **Visão de Processos:** Descreva a execução e interações de fluxos operacionais dentro do sistema, detalhando as principais atividades que acontecem durante a operação.
- **Visão de Implementação:** Mostra a organização do código-fonte em módulos, pacotes e repositórios, garantindo modularidade e futuras expansões.

--	--

- **Visão de Implantação:** Detalhada a infraestrutura física, incluindo nós como servidores e os componentes do sistema, que se integram com as infraestruturas.

Visões	Público	Área	MDS
Caso de Uso	Todos	Realização dos Casos de Usos	Levantamento de Requisitos (Seção - 4.1) Diagramas de Caso de Uso (Seção - 5) Realização de Casos de Uso (Seção - 5.1)
Lógico	Analistas	Estruturas e Classes	Diagrama de Classes (Seção - 6)
Processo	Integradores	Performance Concorrência Fluxos	Levantamento de Requisitos (Seção - 4.2) Diagrama de Sequência e Diagrama de Atividade (Seção - 7)
Implantação	Gerência	Infraestrutura e Nós	Diagrama de Componentes (Seção - 8.0)
Implementação	Programadores	Organização do Código-Fonte	Diagrama de Componentes (Seção - 9.0)

Tabela 3.1

4. Metas e Restrições da Arquitetura

As metas e restrições arquiteturais do sistema definem os padrões de qualidade, segurança e conformidade que devem ser atendimentos pela solução. Elas são derivadas dos requisitos não funcionais levantado no Documento de Visão e alinhada às normas internacionais de engenharia de software (ISO/IEC, IEEE, ITIL, W3C, CMMI). Essas diretrizes orientam as decisões tecnológicas, a priorização de funcionalidades e a gestão dos recursos disponíveis.

Para a representação e documentação da arquitetura do sistema, adota-se a Unified Modeling Language (UML) como linguagem padrão de modelagem, em conformidade com as diretrizes impostas ao OMG e as boas práticas da Engenharia de Software (IEEE). As ferramentas utilizadas para produção de diagramas são LucidChar, StarUML e Draw.io, conforme especificado na tabela 1.2.

4.1. Requisitos Funcionais

ID / Referência	Requisito	Ator(es)	Descrição
RF-01 (REQ-PAN-001)	Cadastro de Usuários	Cliente / Administrador	Permitir cadastro, edição e login seguro de usuários, com autenticação por e-mail/senha e integração

--	--

			com redes sociais (Google, Meta).
RF-02 (REQ-PAN-002)	Autenticador	Cliente / Administrador	Disponibilizar autenticação via e-mail e senha, com suporte a múltiplos fatores de segurança (MFA/2FA) e verificação de credenciais.
RF-03 (REQ-PAN-003)	Nível de Acesso	Administrador	Controlar níveis de acesso diferenciados, restringindo funcionalidades sensíveis (relatórios, dados de vendas) a usuários autorizados.
RF-04 (REQ-PAN-004)	Gerenciamento de Catálogo	Administrador	Permitir cadastro, edição e exclusão de produtos do catálogo, incluindo nome, título, editora, autor e preço.
RF-05 (REQ-PAN-005)	Lista de Desejos	Cliente	Permitir que o usuário crie e gerencie uma lista de produtos de interesse, adicionando ou removendo itens.
RF-06 (REQ-PAN-006)	Gerenciamento de Promoções	Administrador	Criar e gerenciar promoções, cupons de desconto e kits promocionais com data e valor configuráveis.
RF-07 (REQ-PAN-007)	Compra Online	Cliente	Permitir adicionar produtos ao carrinho e concluir compra online, com emissão automática de comprovante de compra.
RF-08 (REQ-PAN-008)	Suporte ao Cliente	Cliente	Disponibilizar canais de suporte: formulário, chatbot e atendimento humano via e-mail ou mensagem.
RF-09 (REQ-PAN-009)	Recuperação de Senha	Cliente	Permitir redefinição de senha por meio de verificação segura por e-mail (link de recuperação).
RF-10 (REQ-PAN-010)	Controle de Vendas	Administrador	Monitorar transações em tempo real, acompanhar status comerciais e gráficos de vendas mensais.
RF-11 (REQ-PAN-011)	Gerenciamento de Pedidos	Administrador	Gerenciar pedidos com atualização de status, registro de problemas

--	--

			de entrega e visualização de clientes.
RF-12 (REQ-PAN-012)	Gerenciamento de Entrega	Cliente / Administrador	Permitir acompanhamento e gerenciamento do status de entrega dos pedidos, com gestão de endereços e seleção de transportadora.
RF-13 (REQ-PAN-013)	Método de Pagamento	Cliente	Fornecer métodos de pagamento: cartão de crédito/débito, boleto bancário e PIX, com integração PagBank e OpenPix.
RF-14 (REQ-PAN-014)	Integração com Redes Sociais	Cliente	Permitir login e compartilhamento em redes sociais (Google, Meta) como funcionalidade nativa.
RF-15 (REQ-PAN-015)	Rastreamento de Pedidos	Cliente	Permitir rastreamento do trajeto de entrega em tempo real via código fornecido pela transportadora.
RF-16 (REQ-PAN-016)	Consulta do Catálogo	Cliente	Pesquisa de produtos com filtros avançados, sugestões de produtos similares, exibição de promoções e mais vendidos.
RF-17 (REQ-PAN-017)	Gerenciar Avaliação	Cliente	Permitir avaliação de produtos adquiridos com nota, comentário e mídia (foto/vídeo).
RF-18 (REQ-PAN-018)	Gestão de Notificações	Cliente / Sistema	Enviar notificações de promoções e lançamentos apenas com consentimento prévio (LGPD, Art. 7º), baseadas em preferências e dados de navegação.
RF-19 (REQ-PAN-019)	Histórico de Pedidos	Cliente / Administrador	Visualizar histórico de pedidos com status, valor, quantidade e data de compra; filtros por período.
RF-20 (REQ-PAN-020)	Relatório de Vendas	Administrador	Gerar e visualizar relatórios de vendas com métricas estratégicas, exportáveis em PDF e CSV.

Tabela 4.1

4.2. Requisitos Não-Funcionais

--	--

ID / Referência	Requisito	Descrição
RNF-21 (RNF-PAN-021)	Responsividade	O sistema deverá ser responsivo e acessível em dispositivos desktop e mobile.
RNF-22 (RNF-PAN-022)	Desempenho em Processamento	O sistema deverá processar orçamentos e pagamentos com rapidez, mesmo em cenários de múltiplos usuários.
RNF-23 (RNF-PAN-023)	Autenticação Segura	O sistema deverá realizar autenticação por e-mail e senha com criptografia de ponta a ponta (HTTPS/TLS 1.2+).
RNF-24 (RNF-PAN-024)	Alta Disponibilidade	O sistema deverá estar disponível 99,5% do tempo.
RNF-25 (RNF-PAN-025)	Usabilidade da Interface	A interface do usuário deverá ser intuitiva e fácil de usar para pessoas de diferentes idades e níveis de familiaridade com a tecnologia.
RNF-26 (RNF-PAN-026)	Escalabilidade em Picos de Acesso	O sistema deverá suportar picos de acesso simultâneo em lançamentos de álbuns e coleções.
RNF-27 (RNF-PAN-027)	Tempo de Carregamento	O tempo médio de carregamento das páginas não deverá exceder 2 segundos.
RNF-28 (RNF-PAN-028)	Compatibilidade entre Navegadores	O sistema deverá ser compatível com os principais navegadores (Chrome, Firefox, Safari, Edge)..
RNF-29 (RNF-PAN-029)	Integração com Pagamentos	O sistema deverá oferecer integração com meios de pagamento seguros (Pix, cartões, PayPal, entre outros).
RNF-30 (RNF-PAN-030)	Recuperação de Falhas	O sistema deve realizar backup diário e obter um plano de recuperação em até 2 horas em caso de falha
RNF-31 (RNF-PAN-031)	Modularidade do Sistema	O sistema deverá ser modular e permitir expansão para novas funcionalidades sem impacto crítico.

--	--

RNF-32 (RNF-PAN-032)	Monitoramento de Desempenho	O sistema deverá possuir ferramentas de monitoramento contínuo de desempenho, garantindo métricas de uso e alertas preventivos.
RNF-33 (RNF-PAN-033)	Personalização de Perfil	O sistema deverá disponibilizar a personalização de perfil do usuário (idiomas, notificações, preferências, entre outros).
RNF-34 (RNF-PAN-034)	Suporte Técnico	O sistema deverá garantir que o suporte técnico esteja disponibilizado 24 / 7, com tempo de resposta máxima estimado em 2 horas.
RNF-35 (RNF-PAN-035)	Proteção de Dados	O sistema deverá estar em conformidade com a LGPD /GDPR, garantindo confidencialidade das informações de usuários.
RNF-36 (RNF-PAN-036)	Acessibilidade	O sistema deverá seguir as diretrizes WCAG 2.1 nível AA para garantir acesso inclusivo a todos os usuários.

Tabela 4.2

5. Visão de Casos de Uso

Esta seção apresenta os casos de uso que representam as funcionalidades centrais do sistema desenvolvido. Os casos de uso selecionados abrangem os principais fluxos arquiteturais da aplicação, envolvendo as operações realizadas pelos clientes e administradores, bem como os componentes responsáveis pelo processamento de informações e persistência dos dados.

5.1. REALIZAÇÕES DE CASO DE USO

A tabela abaixo lista todos os casos de uso do sistema, com suas descrições e rastreabilidade com os requisitos funcionais.

Casos de Uso	Descrição	Requisitos Relacionados
UC01 – Cadastro de Usuários	Permite ao cliente realizar cadastro e autenticação na plataforma.	REQ-PAN-001
UC02 – Autenticação	Permite validar as credenciais de acesso do usuário.	REQ-PAN-002
UC03 – Nível de Acesso	Permite controlar permissões de acesso conforme o perfil do usuário.	REQ-PAN-003
UC04 – Gerenciamento de Catálogo	Permite ao administrador cadastrar, editar e excluir produtos.	REQ-PAN-004

--	--

UC05 – Lista de Desejos	Permite ao cliente gerenciar sua lista de produtos desejados.	REQ-PAN-005
UC06 – Gerenciamento de Promoções	Permite ao administrador criar e gerenciar promoções.	REQ-PAN-006
UC07 – Compra Online	Permite ao cliente realizar compras pela plataforma.	REQ-PAN-007
UC08 – Suporte ao Cliente	Permite ao cliente entrar em contato com o suporte.	REQ-PAN-008
UC09 – Recuperação de Senha	Permite recuperar ou redefinir a senha de acesso.	REQ-PAN-009
UC10 – Controle de Vendas	Permite monitorar e acompanhar as vendas realizadas.	REQ-PAN-010
UC11 – Gerenciamento de Pedidos	Permite ao administrador gerenciar pedidos.	REQ-PAN-011
UC12 – Gerenciamento de Entrega do Produto	Permite controlar informações relacionadas à entrega dos pedidos.	REQ-PAN-012
UC13 – Método de Pagamento	Permite selecionar e processar formas de pagamento.	REQ-PAN-013
UC14 – Integração com Redes Sociais	Permite autenticação utilizando redes sociais.	REQ-PAN-014
UC15 – Rastreamento de Pedidos	Permite acompanhar o status da entrega dos pedidos.	REQ-PAN-015
UC16 – Consulta do Catálogo	Permite pesquisar e visualizar produtos disponíveis.	REQ-PAN-016
UC17 – Gerenciar Avaliação	Permite avaliar produtos adquiridos.	REQ-PAN-017
UC18 – Gestão de Notificações	Permite o envio de notificações aos usuários.	REQ-PAN-018
UC19 – Histórico de Pedidos	Permite consultar pedidos realizados anteriormente.	REQ-PAN-019
UC20 – Relatório de Vendas	Permite gerar relatórios estratégicos de vendas.	REQ-PAN-020

Tabela 5.1

Os diagramas, a seguir, apresentam o modelo completo de casos de uso do sistema, organizadores por atores.

- **Cliente:** Ator principal responsável pela utilização de funcionalidades da plataforma, incluindo cadastro, autenticação, consulta ao catálogo, gerenciamento de lista de desejos; realização de compras, acompanhamento de pedidos, gerenciamento de entregas, avaliações de produtos, recebimento de notificações e solicitação de suporte.
- **Administrador:** Ator responsável por atividades de gerenciamento do sistema, incluindo controle de acesso, gerenciamento do catálogo de produtos, promoções, pedidos, controle de vendas e geração de relatórios gerenciais.

--	--

- **Serviços Externos:** Atores secundários responsáveis pela integração com serviços externos utilizados pela aplicação, tais como autenticação por redes sociais (Google e Meta), processamento de pagamentos (PagBank e OpenPix) e demais serviços de apoio ao funcionamento do sistema.
- **Banco de Dados:** Componentes externos responsáveis pelo armazenamento, consulta e atualização das informações do sistema, sendo utilizado por diversos casos de uso para persistência de dados.

1.1. Diagrama Geral de Caso de Uso

Figura 1. Diagrama de Caso de Uso – Caso de Uso Geral

UC GERAL

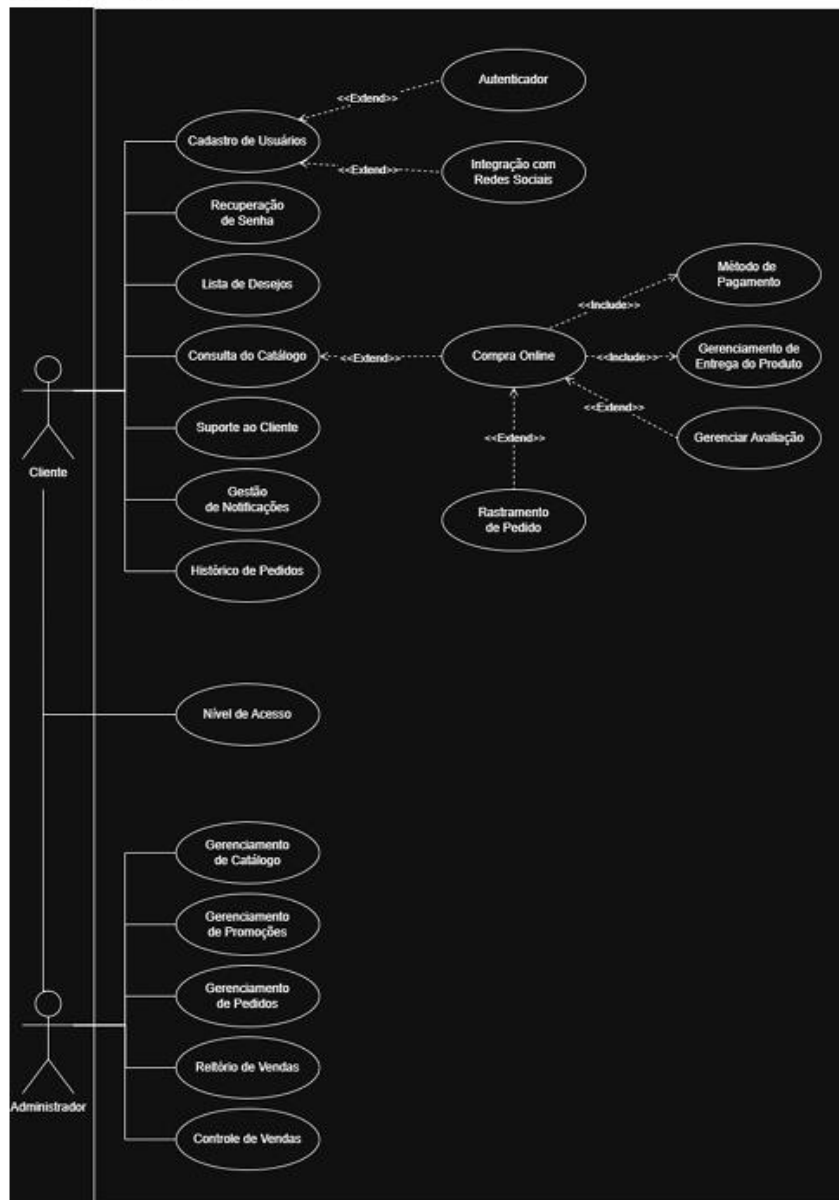


Figura 5.2.1

1.2. Realizações Individuais de Caso de Uso

Esta seção apresenta a realização dos principais casos de uso do sistema, demonstrando como os componentes da arquitetura interagem para executar cada funcionalidade.

UC01 – Cadastro de Usuários

--	--

O processo de cadastro inicia quando o cliente acessa a funcionalidade “**Cadastrar Conta**”. O usuário pode optar por realizar o cadastro manualmente ou utilizar uma conta externa por meio das opções “Cadastrar com Conta Google” ou “**Cadastrar com Rede Social**”. Após o envio das informações, o sistema valida os dados informados e registra a nova conta.

Caso ocorra alguma inconsistência durante o processo, a funcionalidade “**Retornar Erro**” é acionada para informar o problema ao usuário. Após a conclusão do cadastro, o cliente poderá utilizar a funcionalidade “**Entrar com uma Conta**” para acessar o sistema.

Figura 1. Diagrama de Caso de Uso – Cadastro de Usuários

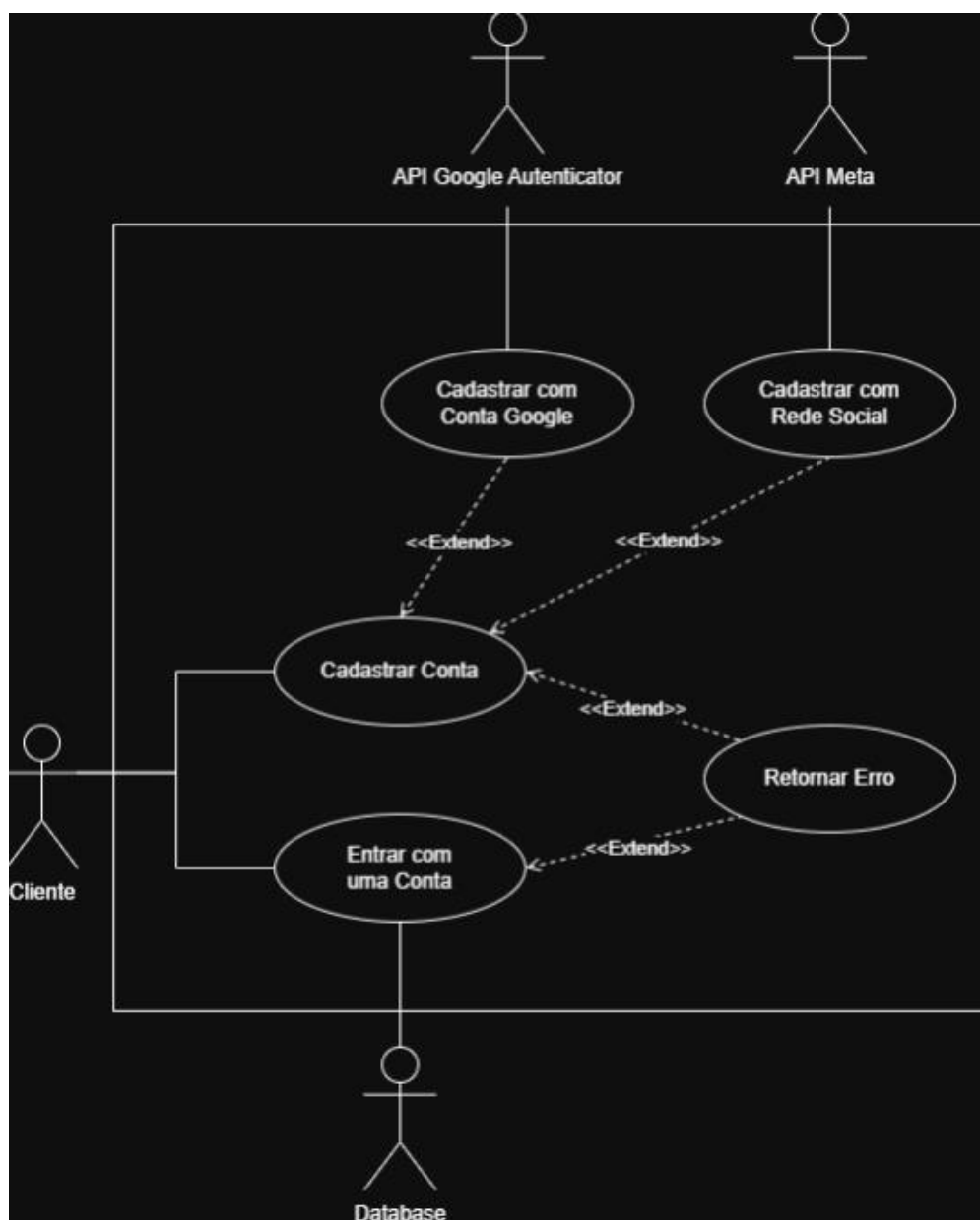


Figura 5.2.1

--	--

UC04 – Gerenciamento de Catálogo

O gerenciamento do catálogo é realizado pelo administrador através da funcionalidade “**Gerenciar Produtos**”. A partir dela, o administrador pode “**Adicionar Novo Produto**”, “**Alterar Produto Existente**” ou “**Excluir Produto Existente**”. Durante o cadastro de um novo produto, são informadas características como nome, título, editora, autor e preço. Após o preenchimento das informações, o sistema executa a funcionalidade “**Armazenar Produto**”, registrando os dados no banco de dados e disponibilizando o produto para consulta no catálogo da plataforma

Figura 2. Diagrama de Caso de Uso – Gerenciamento de Catálogo

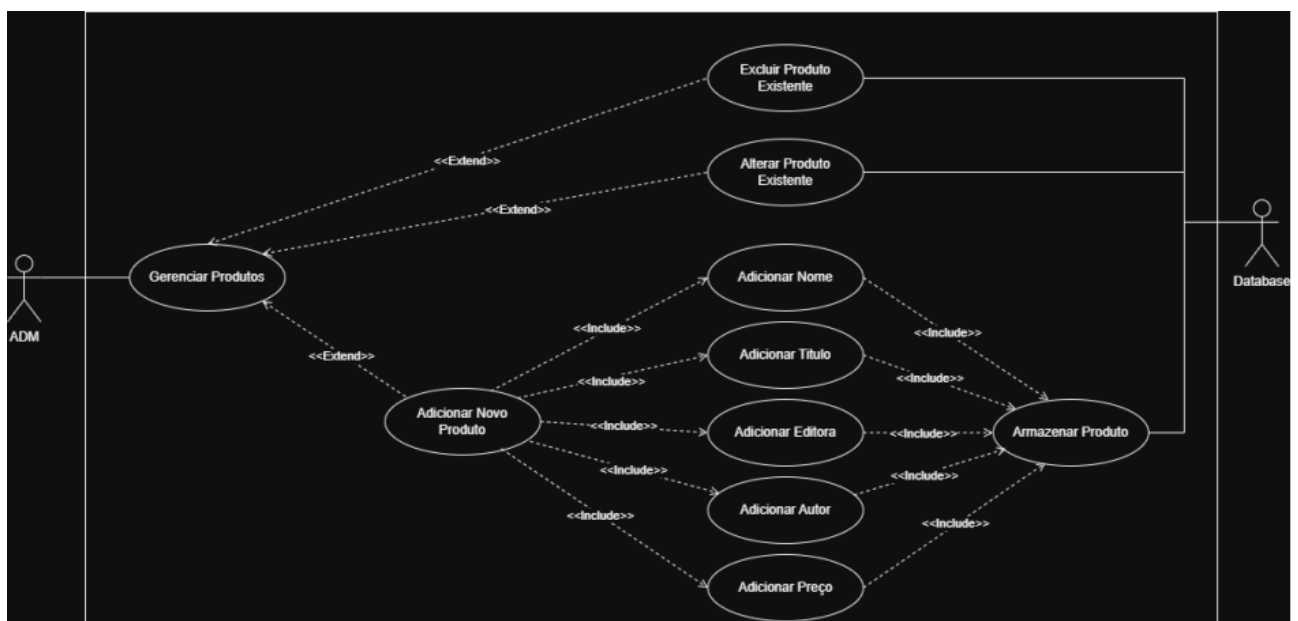


Figura 5.2.2

UC07 – Compra Online

O processo de compra inicia quando o cliente seleciona a funcionalidade “**Realizar Compra do Produto**”. Durante a compra, o usuário deve “**Escolher Método de Pagamento**”, etapa obrigatória para a conclusão da transação. Após a seleção da forma de pagamento, o sistema executa a funcionalidade “**Verificar Pagamento**”, comunicando-se com a API PagBank para validar a operação financeira. Caso a validação seja concluída com sucesso, a compra é finalizada. Em situações de falha ou rejeição do pagamento, a funcionalidade “**Retornar Erro de Pagamento**” é acionada para informar o problema ao cliente.

Figura 3. Diagrama de Caso de Uso – Compra Online

--	--

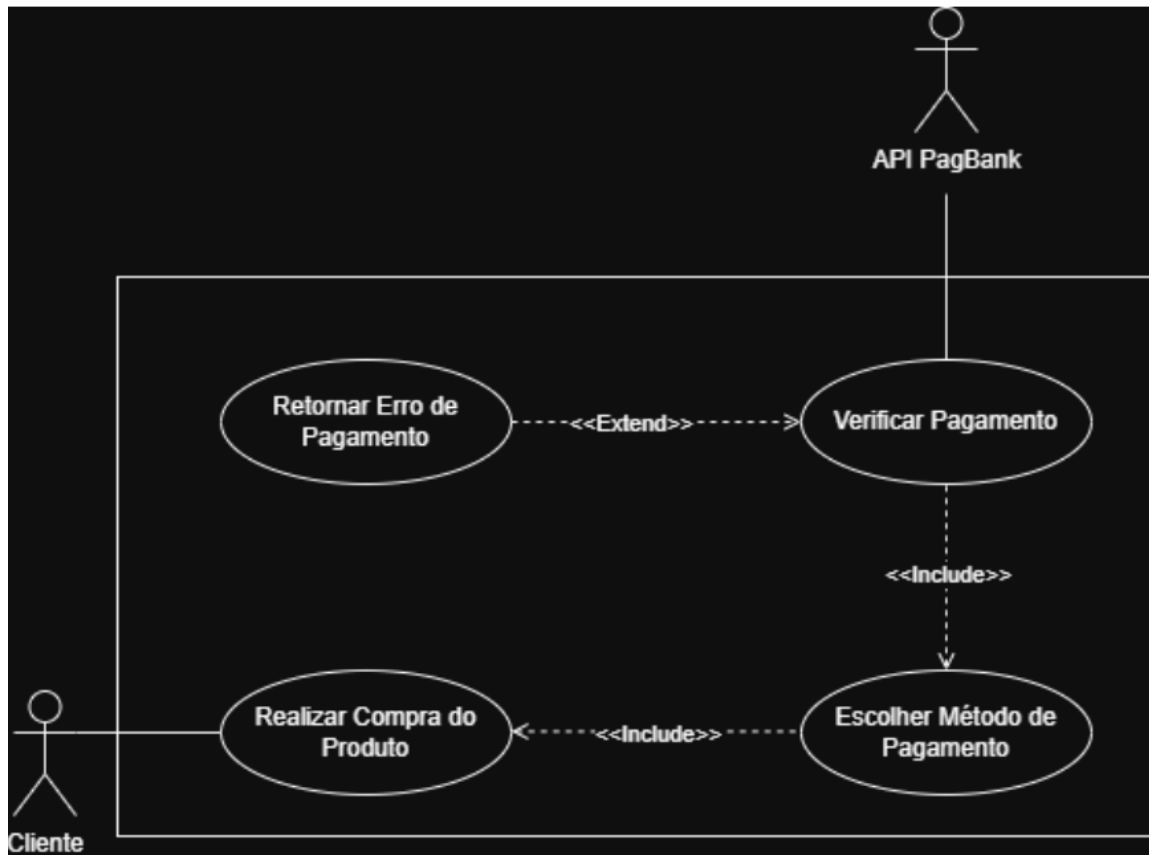


Figura 5.2.3

UC11 – Gerenciamento de Pedidos

O administrador utiliza a funcionalidade “**Gerenciar Pedidos**” para acompanhar as compras realizadas na plataforma. O caso de uso permite “**Visualizar Pedidos**”, consultando informações como número do pedido, nome do cliente, valor total e status da compra. Além disso, o administrador pode utilizar a funcionalidade “**Visualizar Clientes**”, obtendo dados como nome, endereço e histórico dos últimos pedidos realizados. Essas informações auxiliam no acompanhamento e controle das operações de venda e entrega.

Figura 4. Diagrama de Caso de Uso – Gerenciamento de Pedidos

--	--

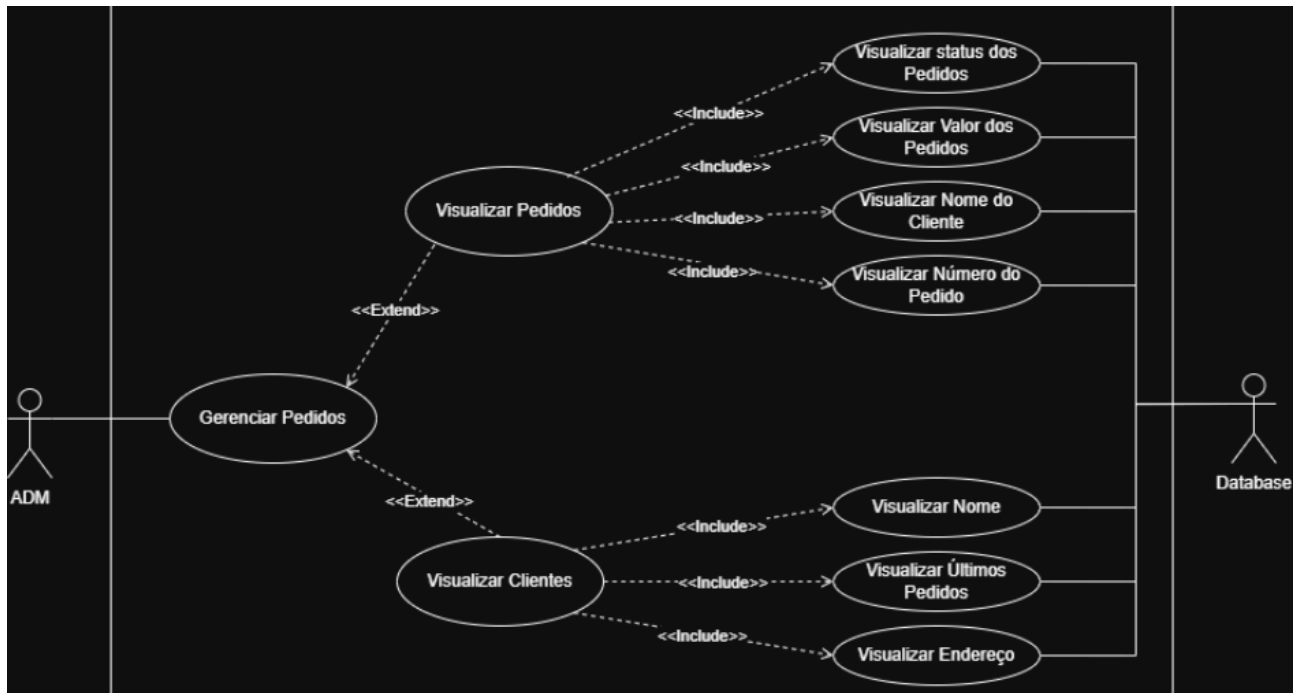
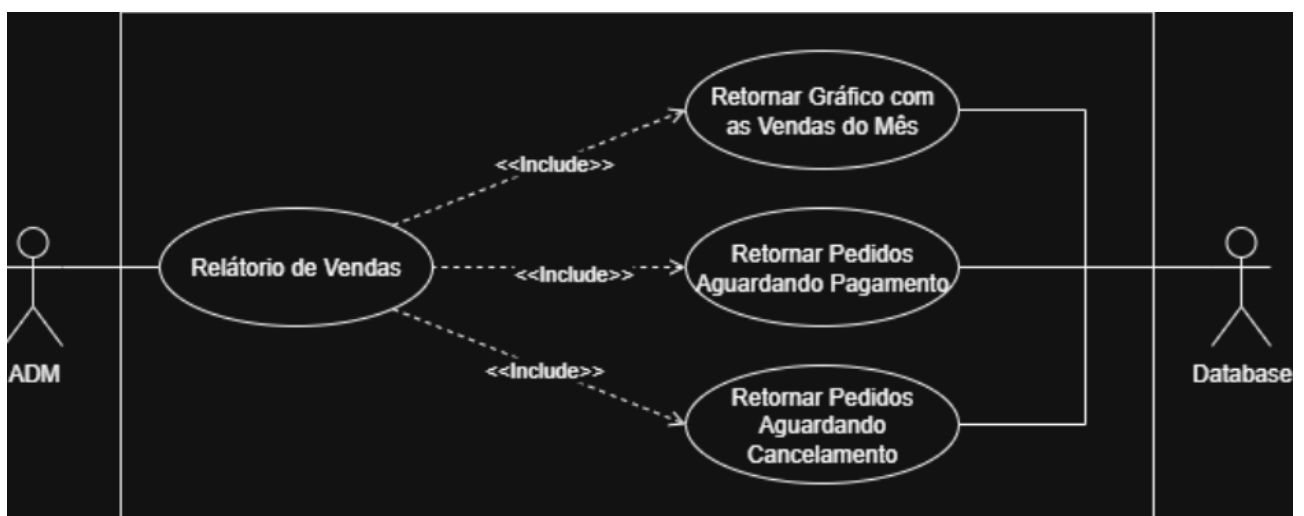


Figura 5.2.4

UC20 - Relatório de Vendas

O administrador acessa a funcionalidade “**Relatório de Vendas**” para acompanhar o desempenho comercial da plataforma. O relatório permite visualizar diferentes indicadores, incluindo o “**Gráfico com as Vendas do Mês**”, os “**Pedidos Aguardando Pagamento**” e os “**Pedidos Aguardando Cancelamento**”. As informações apresentadas auxiliam no monitoramento das vendas e fornecem suporte para a tomada de decisões gerenciais relacionadas ao negócio.

Figura 5. Diagrama de Caso de Uso - Relatório de Venda



--	--

Figura 5.2.5

6. Visão Lógica

Esta seção descreve a organização do modelo de design do Sistema E-commerce Panini, em termos de pacotes, classes e responsabilidades consistentes ao sistema. A arquitetura foi estruturada em camadas, separando as regras de negócios, persistência de dados e entidades de domínio. Visando permitir maior organização, manutenibilidade e escalabilidade de aplicação.

O modelo lógico é composto por pacotes responsáveis pelo gerenciamento de usuários, catálogo de produtos, pedidos, pagamentos, vendas, notificações e demais funcionalidades essenciais do S.E.P.

6.1. Visão Geral

O sistema foi estruturado em pacotes funcionais que representam os principais módulos da plataforma de e-commerce. Cada pacote concentra classes responsáveis por um conjunto específico de funcionalidades.

Figura 1. Diagrama de Classes - Visão Geral de Clases

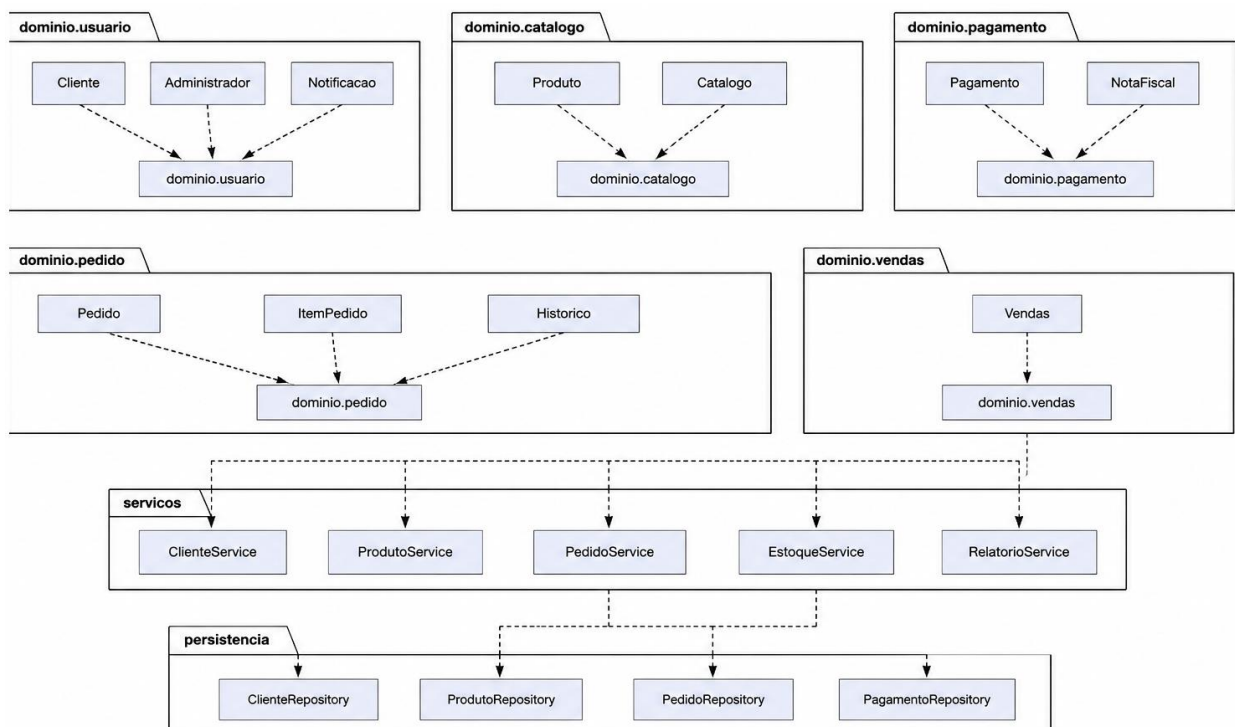


Figura 6.1.1

--	--

Pacote	Descrição
dominio.usuario	Responsável pelo gerenciamento dos usuários da plataforma, incluindo clientes e administradores.
dominio.catalogo	Responsável pelo gerenciamento dos produtos disponibilizados para venda.
dominio.pedido	Responsável pelo processamento dos pedidos realizados pelos clientes.
dominio.pagamento	Responsável pelo processamento dos pagamentos e emissão de documentos fiscais.
dominio.vendas	Responsável pelo controle e análise das vendas realizadas na plataforma.
servicos	Responsável pela implementação das regras de negócio do sistema.
persistencia	Responsável pelo acesso e manipulação dos dados armazenados.

Tabela 6.1.1

6.2. Pacotes de Design da Arquitetura

Esta subseção apresenta os pacotes considerados arquiteturalmente significativos para o Sistema Panini. Para cada pacote é apresentado um diagrama de classes contendo os principais elementos do domínio, seguido da descrição das classes mais relevantes.

Figura 1. Diagrama de Pacotes – Pacotes na Arquitetura

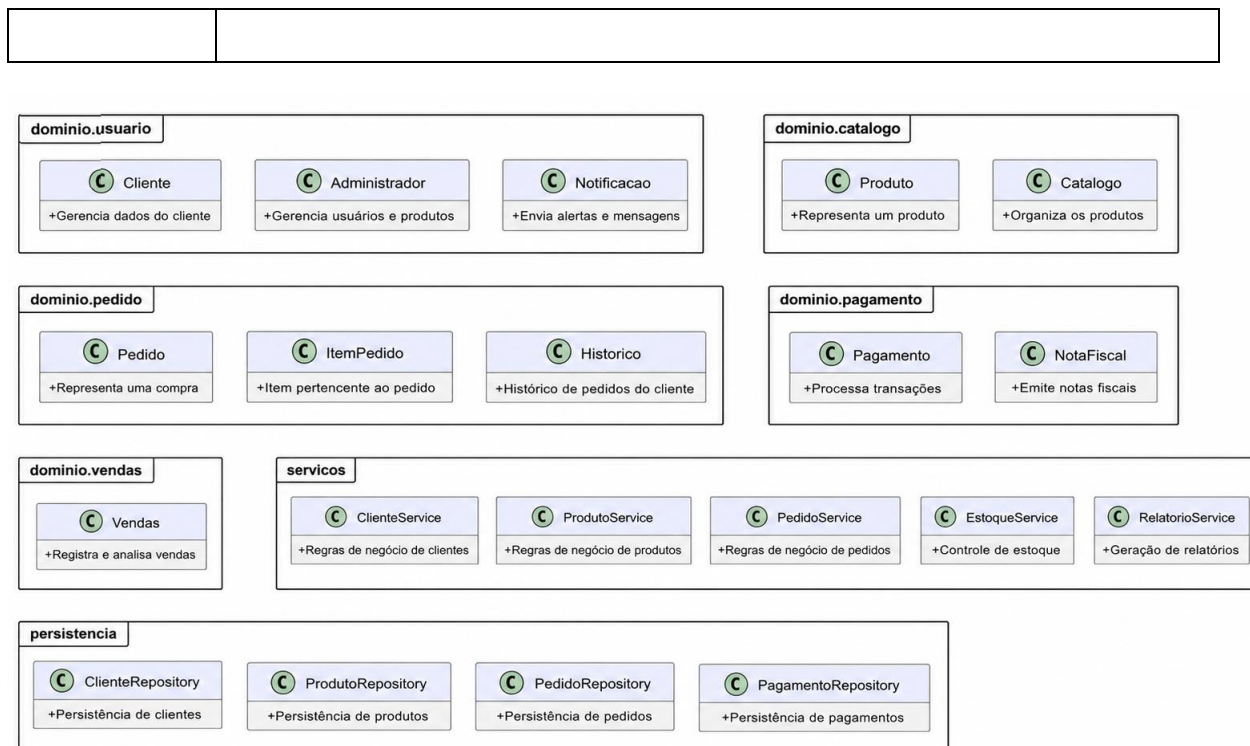


Figura 6.2.1

Pacote: dominio.usuario

O pacote dominio.usuario reúne as classes responsáveis pelo gerenciamento dos usuários do Sistema Panini. Essas classes contemplam as funcionalidades relacionadas ao cadastro e administração dos usuários da plataforma, bem como o envio de notificações referentes às atividades realizadas no sistema.

Figura 2. Diagrama de Pacotes – Pacotes de Domínio do Usuário

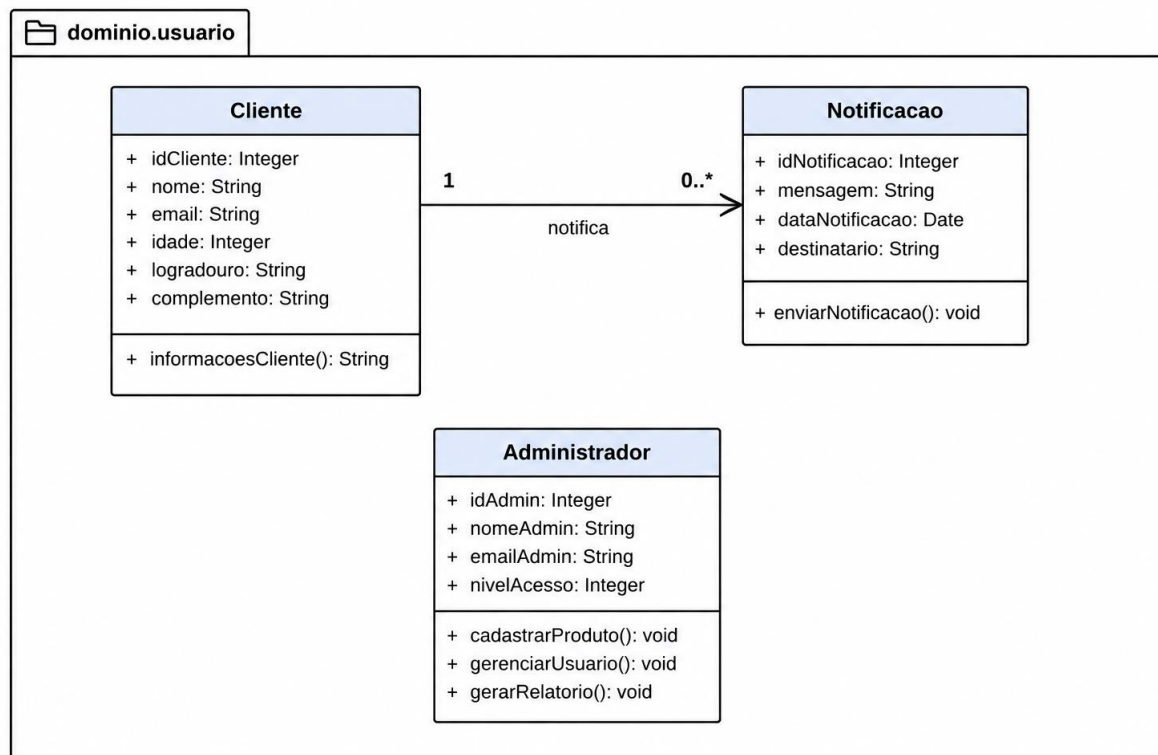


Figura 6.2.2

Pacote: dominio.catalogo

O pacote **dominio.catalogo** é responsável pelo gerenciamento dos produtos disponibilizados no Sistema Panini, contemplando as funcionalidades relacionadas à organização, manutenção e consulta do catálogo utilizado pelos clientes durante o processo de compra

Figura 3. Diagrama de Pacotes – Pacotes de Domínio de Catálogo

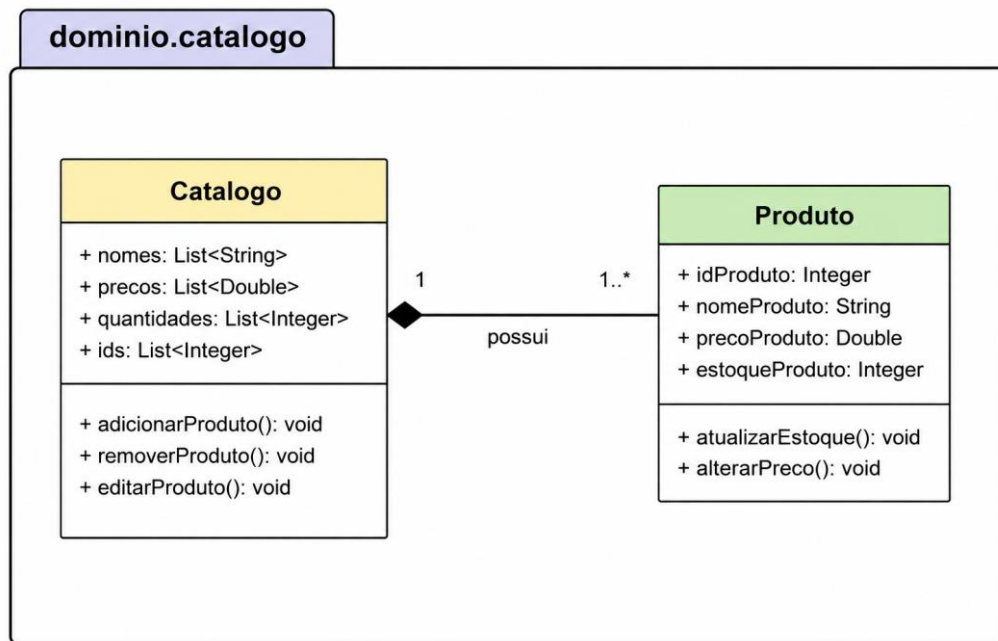


Figura 6.2.3

Pacote: dominio.pedido

O pacote `dominio.pedido` reúne as classes responsáveis pelo gerenciamento dos pedidos realizados pelos clientes, incluindo os itens associados a cada compra e o histórico de pedidos efetuados na plataforma.

Figura 4. Diagrama de Pacotes – Pacotes de Domínio de Pedido

--	--

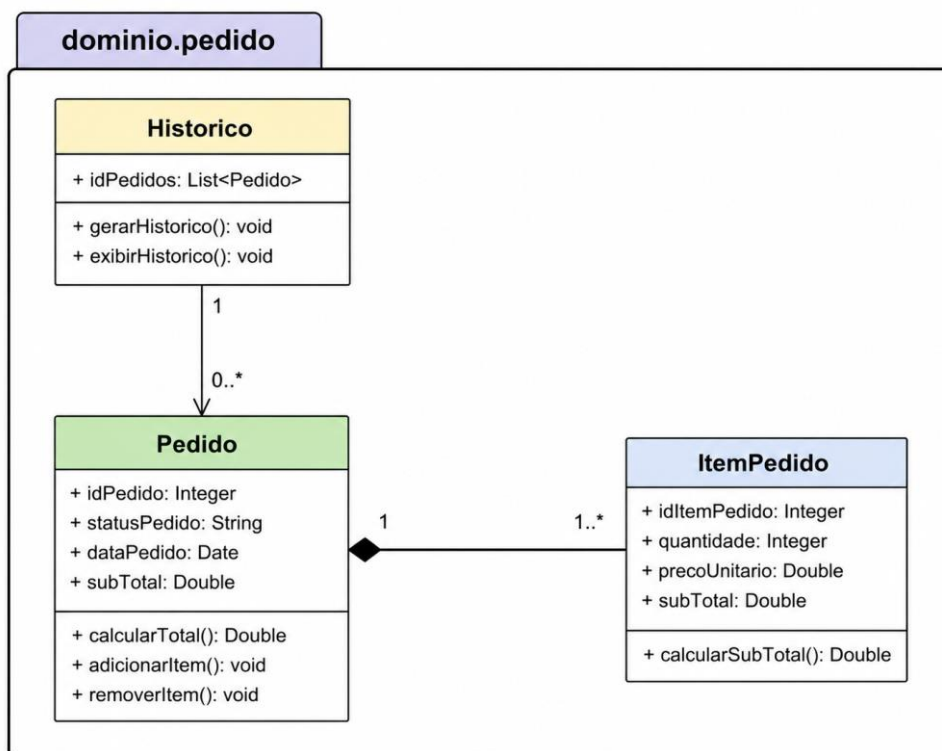


Figura 6.2.4

Pacote: dominio.pagamento

O pacote dominio.pagamento é responsável pelo gerenciamento das transações financeiras realizadas no Sistema Panini, contemplando o processamento dos pagamentos efetuados pelos clientes e a emissão das respectivas notas fiscais associadas às compras realizadas na plataforma.

Figura 5. Diagrama de Pacotes – Pacotes de Domínio de Pagamento

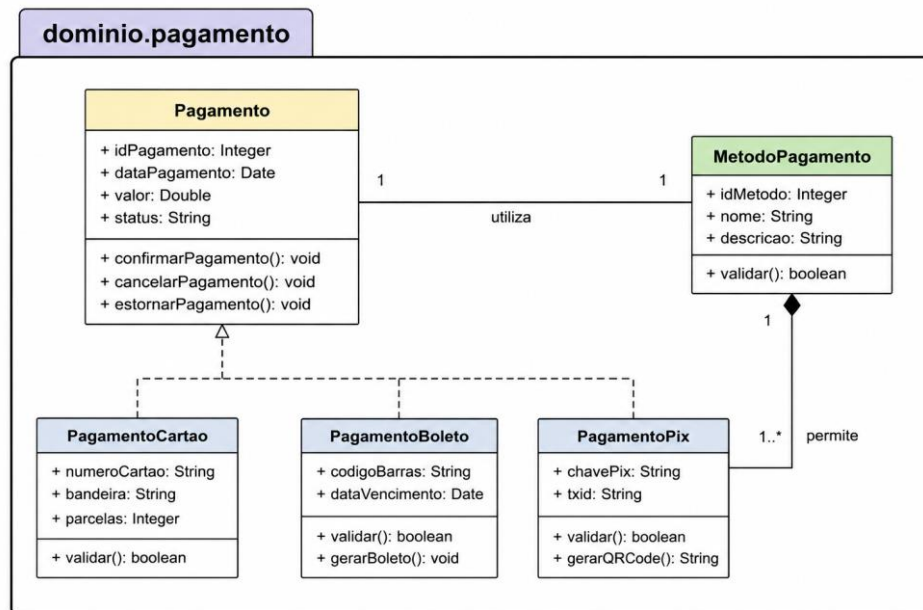
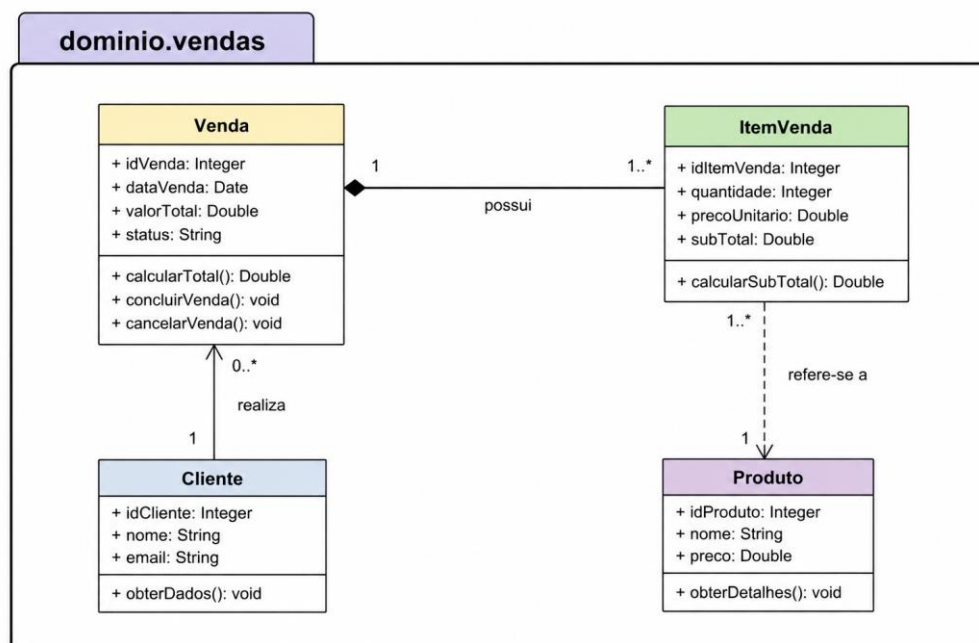


Figura 6.2.5

Pacote: dominio.vendas

O pacote domínio.vendas é responsável pelo registro e acompanhamento das vendas realizadas no Sistema Panini, fornecendo informações que auxiliam no controle das operações comerciais e na geração de relatórios gerenciais para apoio à tomada de decisão administrativa.

Figura 6. Diagrama de Pacotes – Pacotes de Domínio de Vendas



--	--

Figura 6.2.6

6.3. Camadas

A arquitetura S.E.P encontra-se organizada em camadas, de forma a promover a separação de responsabilidades e facilitar a manutenção dos componentes da aplicação. Cada camada possui funções específicas e se comunica com os demais serviços por meio de interfaces definidas.

Camada de Apresentação (View)

A camada de apresentação é responsável pela interação entre os usuários e o sistema, disponibilizando as interfaces necessárias para o acesso às funcionalidades fornecidas pela plataforma.

Figura 1. Diagrama de Classes – Camada de Apresentação

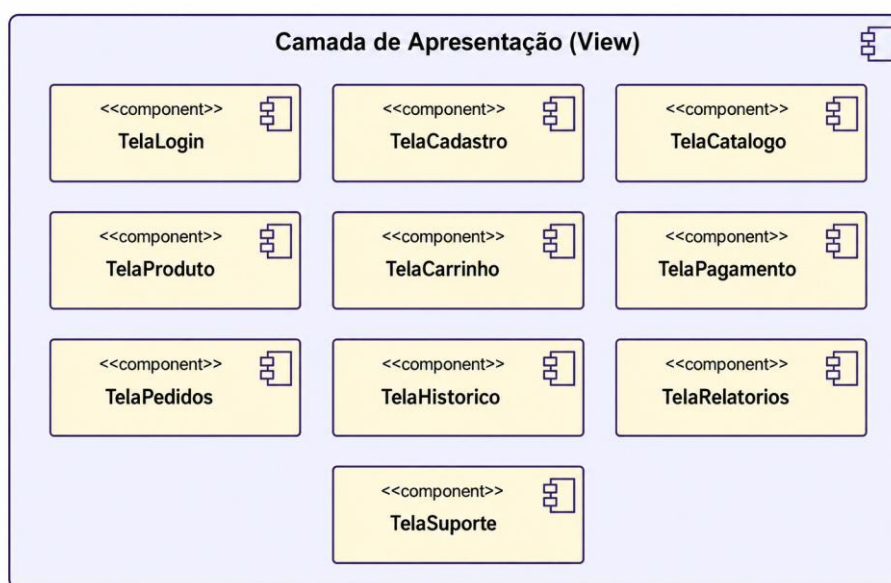


Figura 6.3.1

Camada de Serviços

A camada de serviços concentra na execução de regras de negócios definidas ao sistema. Seus componentes são responsáveis pelo processamento de operações solicitadas aos usuários, pela validação de informações e pela coordenação da comunicação entre as camadas de apresentação, persistência e integrações externas.

Figura 2. Diagrama de Classe – Camada de Serviços

--	--

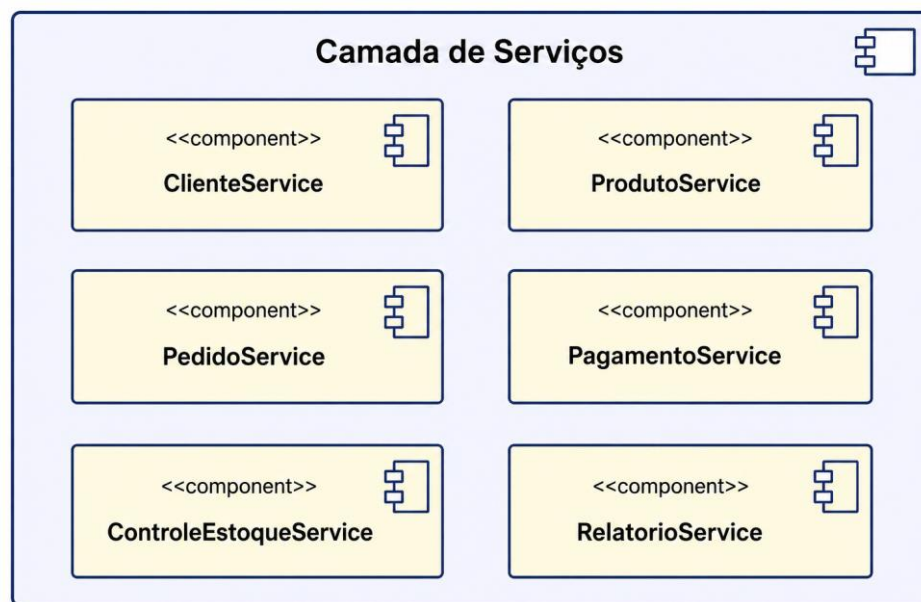
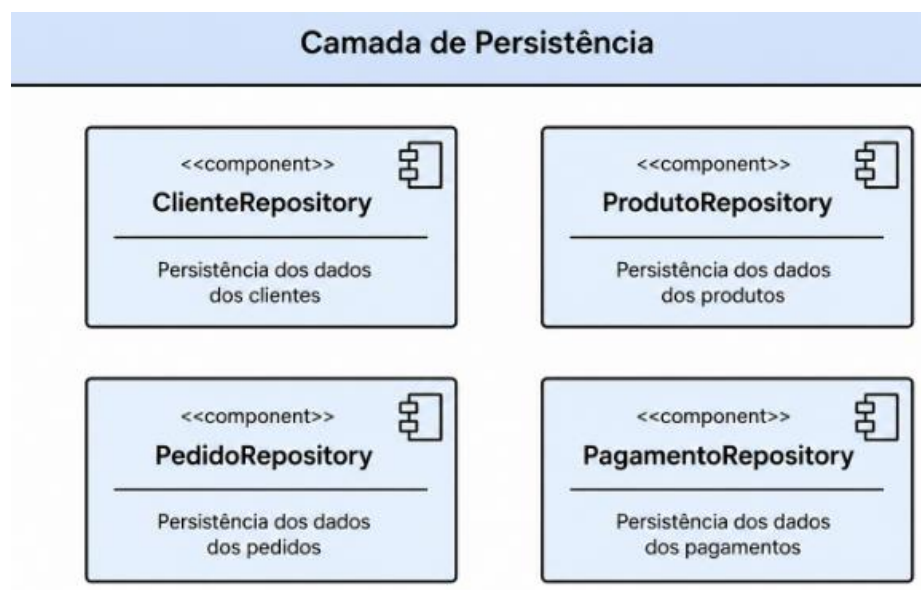


Figura 6.3.2

Camada de Persistência

A camada de persistência é responsável pelo gerenciamento do acesso de dados armazenados pela aplicação. Os componentes desta camada encapsulam as operações de criação, consulta, atualização e remoção de informações necessárias ao funcionamento do sistema, promovendo a separação entre as regras de negócio e os mecanismos de armazenamento. Cada repositório atua sobre um conjunto específico de entidades, contribuindo para a organização e manutenção da arquitetura de software.

Figura 3. Diagrama de Classe – Camada de Persistência



--	--

Figura 6.3.3

Camada de Armazenamento e Integrações Externas

A camada de armazenamento e integrações externas é responsável pelo armazenamento permanente de informações do sistema, e pela comunicação estabelecida entre os serviços externos utilizados pela aplicação. Os componentes desta camada permitem a persistência dos dados da plataforma, a autenticação por provedores externos, o processamento de pagamentos eletrônicos e o envio de notificações aos usuários, garantindo a interoperabilidade necessária ao funcionamento do sistema.

Figura 4 Diagrama de Classe – Camada de Armazenamento e Integração Externa



Figura 6.3.4

7. Visão de Processos

Esta seção descreve a decomposição do sistema de e-commerce em processos leves (threads simples de controle) e processos pesados (agrupamentos de processos leves). Os processos são organizados em grupos que se comunicam de forma síncrona por meio de requisições HTTP/REST com respostas em JSON. A comunicação entre os subsistemas ocorre por transmissão de mensagens, e o sistema utiliza threads para tratar validações e confirmações de maneira paralela quando necessário,

São representados dois processos de negócio principais: “Realizar um Pedido” e “Pagamento e Checkout”. Cada um desses processos é representado por um Diagrama de Sequência, que mostra as interações entre

--	--

os atores e os subsistemas, e um Diagrama de Atividades, que detalha o fluxo de controle e as decisões envolvidas.

PROCESSO 1: REALIZAR UM PEDIDO

O processo de “Realizar um Pedido” envolve quatro participantes: o Cliente, o Sistema Web, o Serviço de Pedidos e o Banco de Dados. O fluxo é iniciado quando o cliente adiciona os produtos ao carrinho e confirma o pedido. O Serviço de Pedidos valida a disponibilidade em estoque junto ao Banco de Dados.

Caso haja disponibilidade, reserva os itens e registra o pedido, retornando ao cliente o resumo da compra. Caso contrário, o cliente é notificado sobre a indisponibilidade do produto.

Figura 1. Diagrama de Sequência - Realizar um Pedido

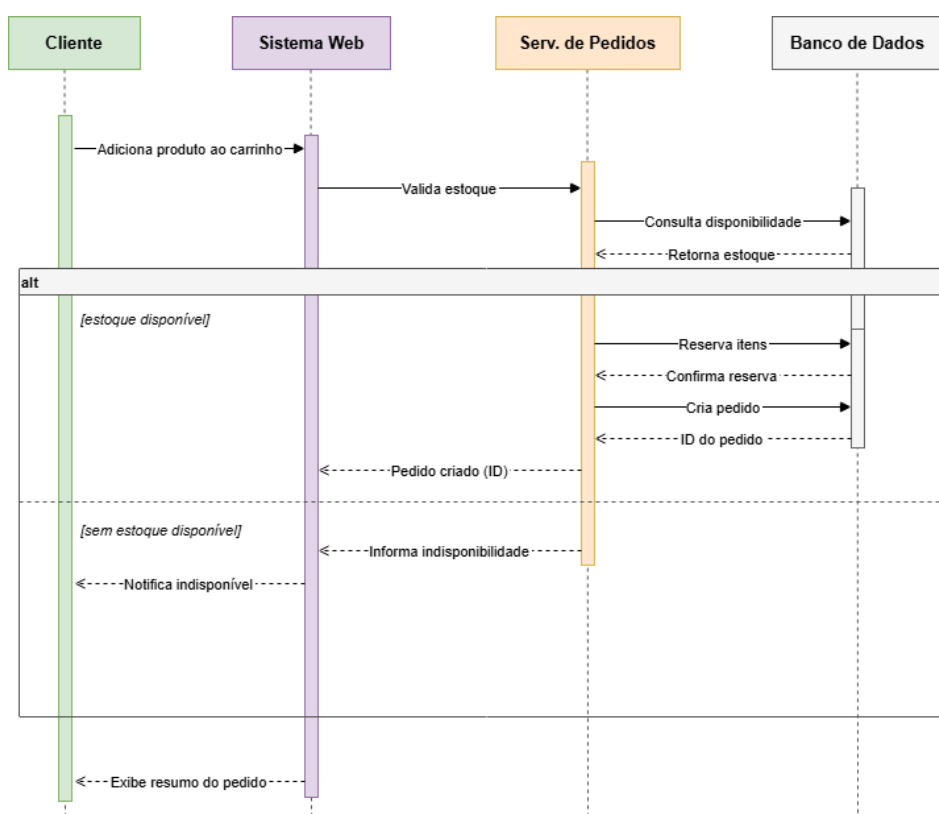


Figura 7.1.1

Figura 2. Diagrama de Atividades – Realizar um Pedido

--	--

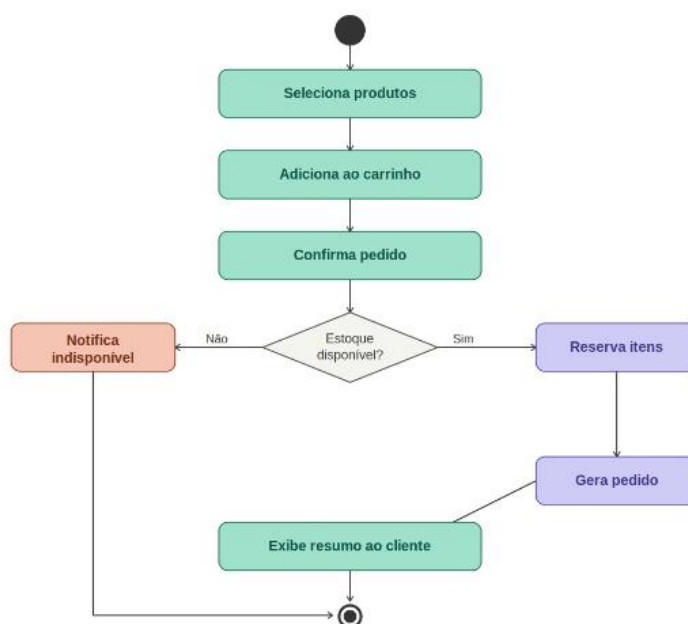


Figura 7.1.2

PROCESSO 2: PAGAMENTO E CHECKOUT

O processo de “Pagamento e Checkout” envolve o Cliente, o Sistema Web, o Gateway de Pagamento e o Banco de Dados. O fluxo começa quando o cliente inicia o checkout e informa os dados de pagamento. O Sistema Web repassa as informações ao Gateway de Pagamento, que processa a transação de forma interna. O resultado (aprovado ou recusado) é retornado ao Sistema Web: em caso de aprovação, o pedido é atualizado no Banco de Dados e a nota fiscal é gerada, com confirmação enviada ao cliente, em caso de recusa, o cliente é notificado da falha.

Figura 3. Diagrama de Sequência - Pagamento e Checkout

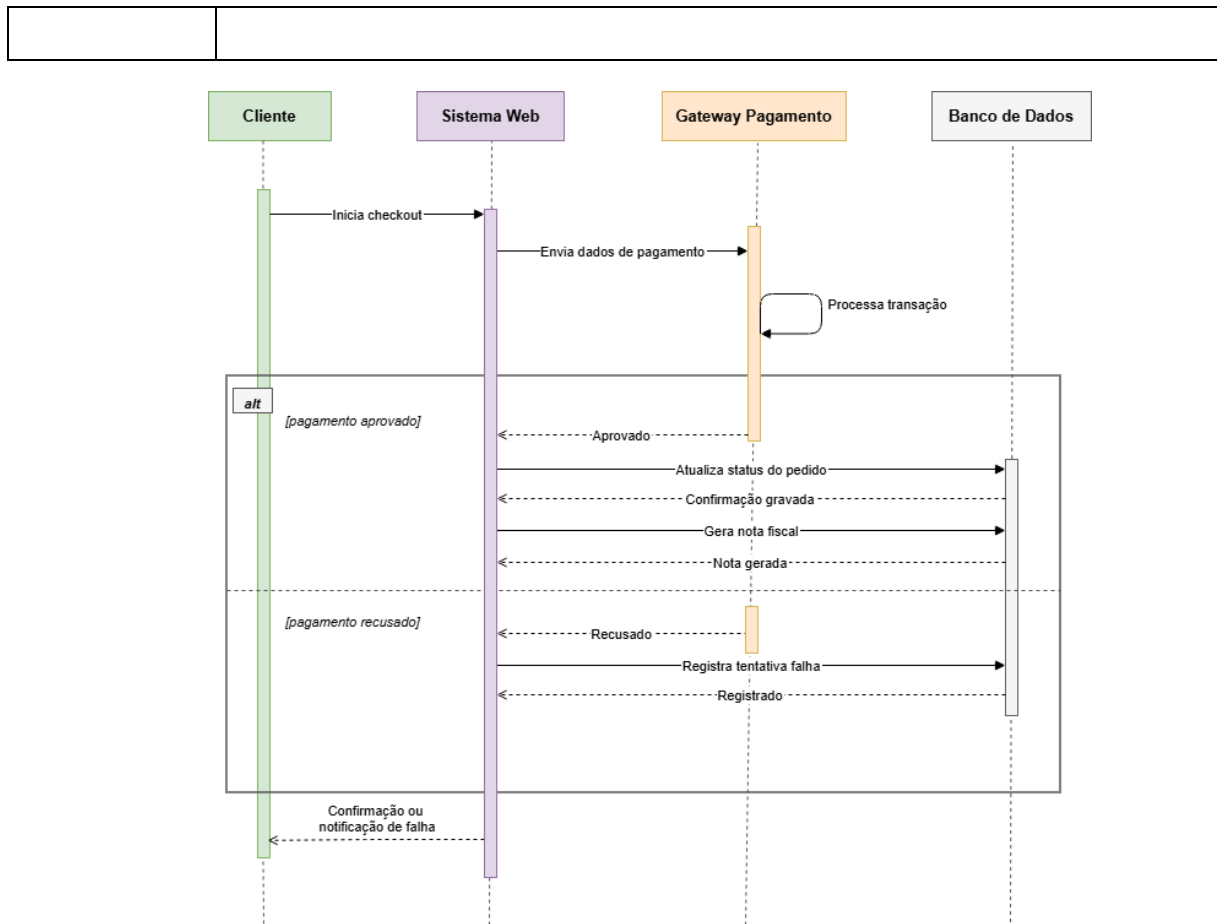
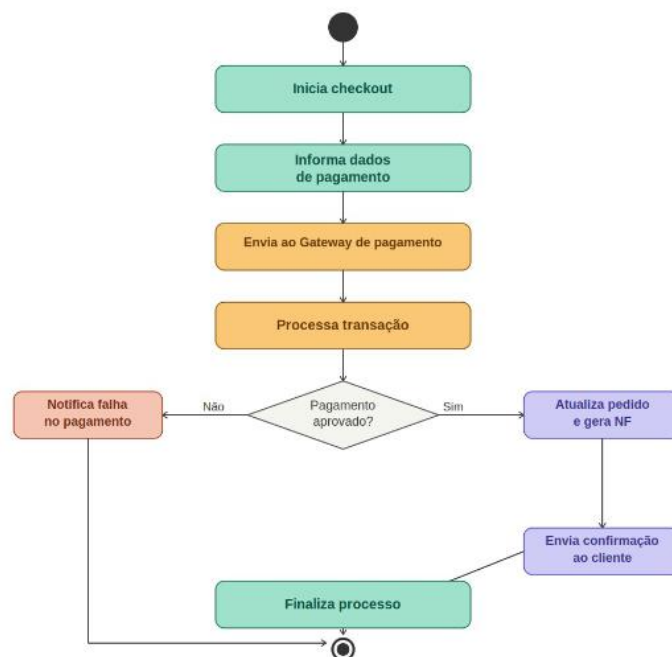


Figura 7.2.3

Figura 4. Diagrama de Atividade – Pagamento e Checkout



--	--

Figura 7.2.4

8. Visão de Implantação

Esta seção descreve a configuração de rede física, na qual o Sistema E-commerce Panini (S.E.P) é implantado e executado. A topologia adotada segue o modelo *client-server* (cliente-servidor) composta por três categorias de nós: o dispositivo do usuário final, o servidor de aplicação centralizado e os dispositivos externos de serviços terceirizados.

Toda a comunicação entre os nós aplicados ocorre exclusivamente por meio do protocolo HTTPS, fornecendo a criptografia em trânsito em conformidade com os requisitos descritos (“RNF-PAN-023 - Autenticação Segura”).

O servidor local do usuário “VPS Linux” atua como o nó central da infraestrutura, hospedando os três artefatos de software internos do sistema: o Front-end, o Back-end (SpringBoot) e o Banco de Dados (MySQL). O Back-end constitui o ponto de integração central da arquitetura, sendo o único componente autorizado a se comunicar diretamente ao banco de dados, através do protocolo de conectividade de banco de dados “JDBC” e os serviços externos de autenticação e validação de pagamento (via HTTPS). O Front-end React interage exclusivamente com a camada lógica através de uma API REST, sem disponibilidade de acesso direto à camada de persistência ou aos demais serviços externos.

Figura 1. Diagrama de Pacotes - Estruturação Geral do Sistema

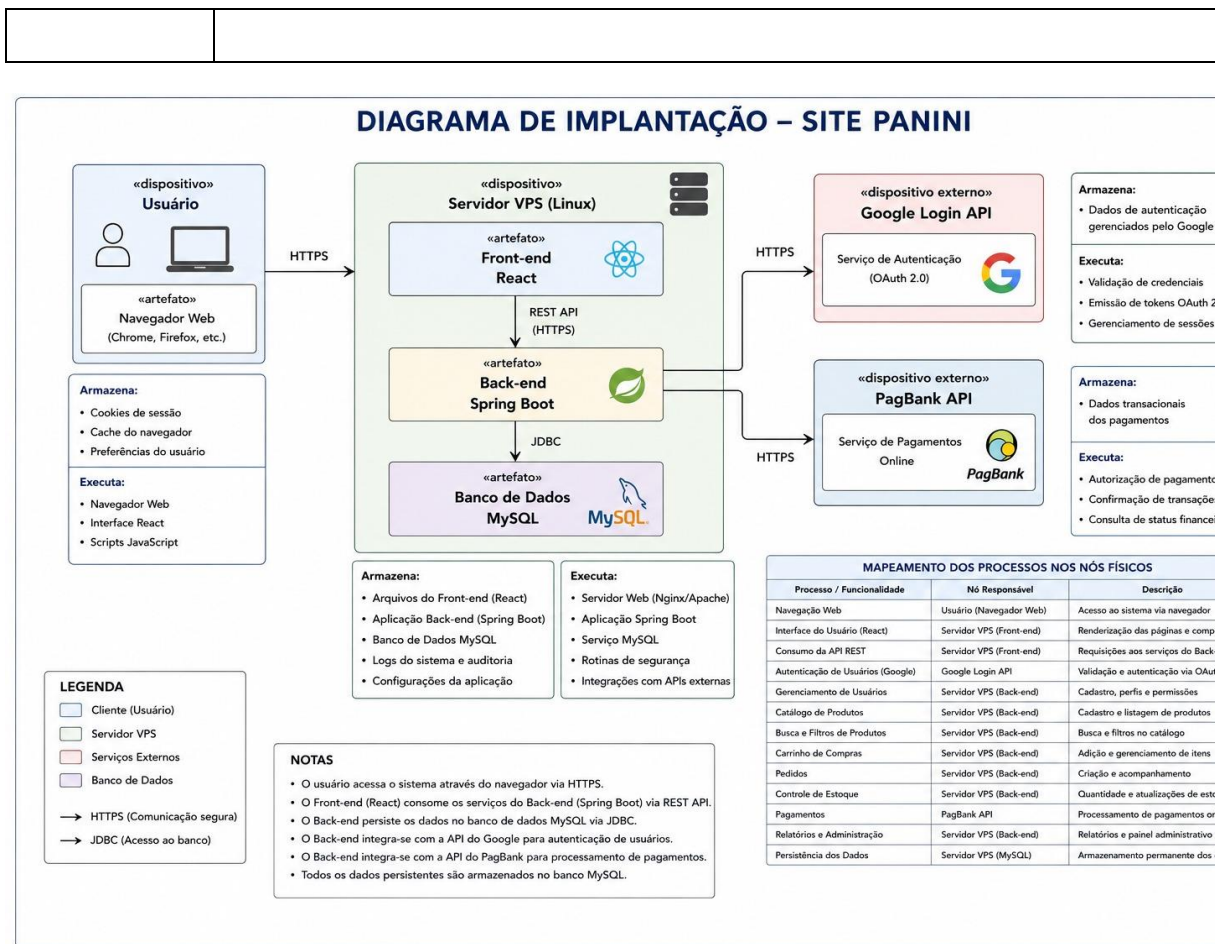


Figura 8.1

8.1. Descrição de Nós-Físicos

A seguir, são descritos os nós-físicos que compõem a infraestrutura de implementação, identificados ao modelo acima:

1. **Usuário (Navegador Web):** Representa o dispositivo do usuário final, que pode ser qualquer dispositivo móvel equipado com um navegador web compatível (Google Chrome, Mozilla Firefox, Microsoft Edge, etc). Este nó não hospeda nenhum componente do sistema, sendo o ponto inicial de acesso ao sistema por meio da interface fornecida pelo React, renderizada ao navegador. A comunicação com o Servidor VPS ocorre exclusivamente via HTTPS.

1.1. **Armazenamento:** O sistema aloca os cookies de sessão, cache de navegação do browser e preferências locais do usuário.

1.2. **Executa:** Execução direta ao navegador web implementado pela interface React (renderização *client-side*) e scripts desenvolvidos em JavaScript.

--	--

2. Servidor VPS Linux: Constitui-se ao nó central da implantação. Trata-se de um servidor virtual privado (VPS) com o sistema operacional Linux, responsável por hospedar e executar todos os componentes de software internos do sistema. Neste nó, estão contidos os três artefatos principais da solução: o Front-end React, o Back-end SpringBoot e o Banco de Dados MySQL. O acesso externo a este nó é gerenciado pelo servidor web Nginx/Apache, que atua como proxy reverso e o ponto de entrada de requisições HTTPS provenientes do dispositivo do usuário.

2.1. Armazenamento: Os arquivos estáticos do Front-end React, Códigos-Fontes compilados da aplicação SpringBoot, dados alocados ao Banco de Dados MySQL, Logs do Sistema e de Auditoria, e Arquivos de Configuração de aplicação.

2.2. Executa: Servidor web Nginx/Apache, aplicação SpringBoot, serviço MySQL e rotinas de segurança e integração com API's externas.

3. Google Login API (Serviço Externo): Dispositivo externo gerenciado pelo Google, responsável pelo serviço de autenticação de usuários via protocolo OAuth 2.0. O serviço é acionado pelo Back-end SpringBoot sempre que um usuário realiza login ou cadastro por meio da integração com redes sociais ("RF-PAN-014 - Integração com Redes Sociais"). A comunicação ocorre via HTTPS, e nenhum dado de autenticação é armazenado diretamente no sistema S.E.P (o gerenciamento de credenciamento é de responsabilidade exclusiva da infraestrutura do Google).

3.1. Armazenamento: Armazena os dados de autenticação e credenciais gerenciados pela infraestrutura e serviço do Google.

3.2. Executa: Validação de credenciais OAuth 2.0, emissão de tokens de acesso e gerenciamento de sessões autenticadas.

4. PagBank API (Serviço Externo): Dispositivo externo gerenciado pelo fornecedor "PagBank", responsável pelo gerenciamento de transações financeiras do sistema. O recurso é acionado pelo Back-end SpringBoot durante o fluxo de Pagamento e Checkout ("RF-PAN-013 - Método de Pagamento"), recebendo os dados de pagamento e retornando o resultando da transição, definida como "aprovada" ou "reprovada". A comunicação ocorre via HTTPS, e os dados transacionais são armazenados na estrutura do fornecedor, com os registros de confirmação persistidos no Banco de Dados integrado ao sistema.

4.1. Armazenamento: Os dados transacionais e registros financeiros das operações transacionais de pagamento.

--	--

4.2. Executa: A autorização de métodos de pagamentos, confirmação de transações e consulta de status de operações financeiras.

8.2. Mapeamento dos Processos de Nós-Físicos

A tabela, a seguir, apresenta o mapeamento entre os processos funcionais do sistema e nós físicos responsáveis por sua execução, evidenciando como cada funcionalidade se distribui pela topologia de implantação descritiva nesta seção. Este mapeamento estabelece a rastreabilidade entre a Visão de Processos e a Infraestrutura Física, conforme o exigido ao modelo arquitetural Kruchten (4+1).

Processo / Funcionalidade	Nó Responsável	Descrição
Navegação Web	Usuário (Navegador Web)	Acesso ao sistema via navegador, renderização da interface
Interface do Usuário (React)	Servidor VPS (Front-end)	Renderização de páginas e componentes da aplicação
Consumo da API REST	Servidor VPS (Front-end)	Requisições ao Back-end via REST API (HTTPS)
Autenticação de Usuários (Google)	Google Login API	Validação e autenticação via OAuth 2.0
Gerenciamento de Usuários	Servidor VPS (Back-end)	Cadastro, perfis e controle de permissões
Catálogo de Produtos	Servidor VPS (Back-end)	Cadastro e listagem de produtos
Busca e Filtros de Produto	Servidor VPS (Back-end)	Busca e aplicação de filtros no catálogo
Carrinho de Compra	Servidor VPS (Back-end)	Adição e gerenciamento de itens do carrinho
Pedidos	Servidor VPS (Back-end)	Criação e acompanhamento de pedidos
Controle de Estoque	Servidor VPS (Back-end)	Atualização de quantidades disponíveis
Pagamentos	PagBank API	Processamento de pagamentos online
Relatórios e Administração	Servidor VPS (Back-end)	Relatórios gerenciais e painel administrativo
Persistência de Dados	Servidor VPS (MySQL)	Armazenamento permanente de todos os dados do sistema

Tabela 8.2.1

9. Visão de Implementação

Esta seção descreve a estrutura geral do modelo de implementação do Sistema E-commerce Panini (S.E.P), apresentando a organização do software em camadas arquiteturais, os subsistemas que compõem cada camadas, os componentes significativos do ponto de vista da arquitetura e as tecnologias empregadas em cada nível de solução.

--	--

A arquitetura implementada do S.E.P é estruturada em quatro camadas definidas: Camada de Apresentação (Front-end), Camada de Aplicação (Back-end), Camada de Persistência (Banco de Dados) e Serviços Externos. Esta divisão materializa o padrão arquitetural CQRS (Command Query Responsibility Segregation) adotado ao projeto, onde as operações de leitura (Queries) e escrita (Command) trafegam por subsistemas distintos dentro da Camada de Aplicação, garantindo separação de responsabilidades, rastreabilidade de operações e capacidade de escalabilidade horizontal.

As fronteiras entre as camadas são rígidas e controladas: o Front-end comunica-se com o Back-end exclusivamente via REST API sobre HTTPS, o Back-end acessa a camada de persistência exclusivamente com a conectividade via JDBC (Java Database Connection), e os Serviços Externos são acionados apenas pelo Back-end, permitindo o acesso de recursos diretamente pelo banco de dados. Essa arquitetura assegura isoladamente cada responsabilidade, facilitando a manutenção evolutiva do sistema em conformidade com as funcionalidades requisitadas (“RNF-PAN-O31 – Modularidade do Sistema”).

Figura 1. Diagrama de Pacotes – Estrutura Geral entre Camadas

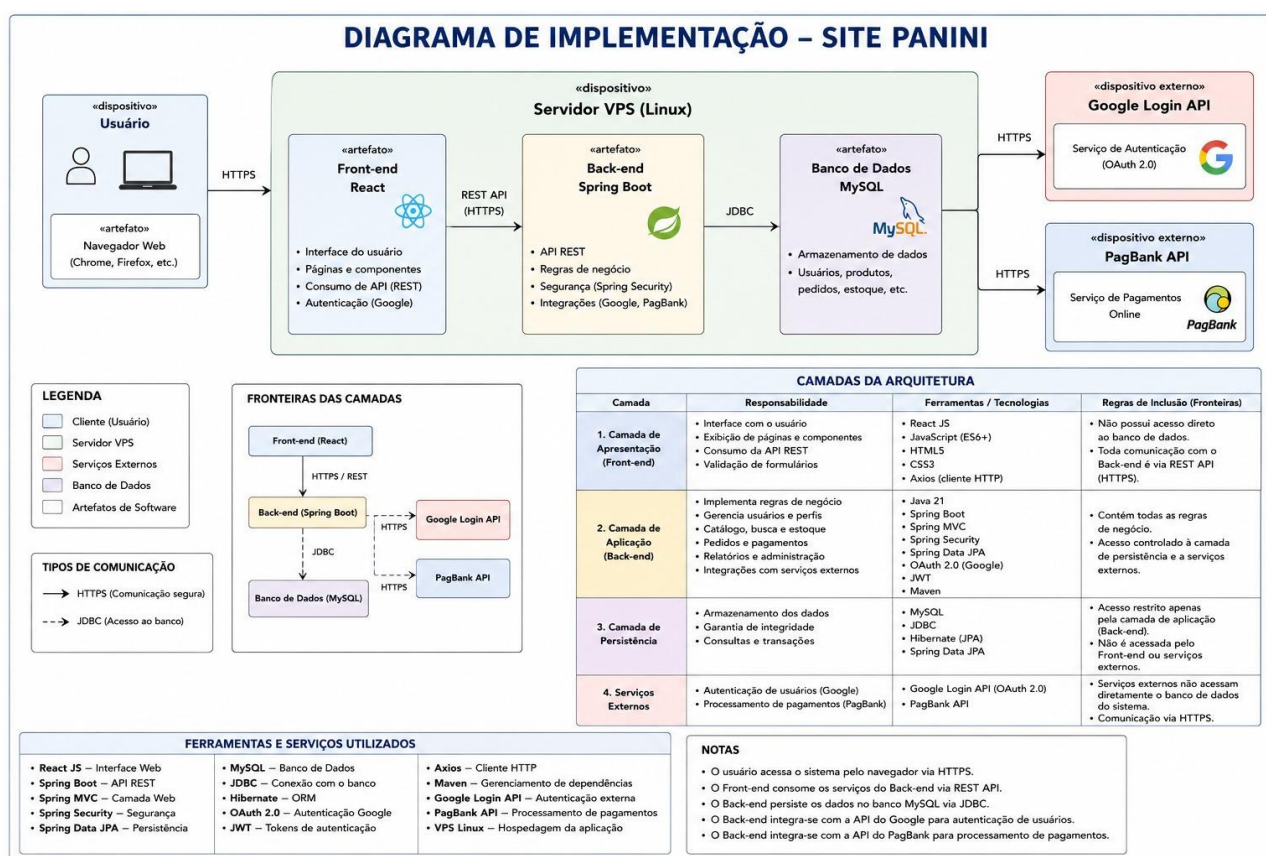


Figura 9.1

9.1. Visão Geral

Este modelo é organizado em camadas arquiteturais apresentadas conforme representada pelo diagrama.

--	--

- Camada Visual (Frontend): É a área responsável por toda a interação direta com o usuário final. A camada renderiza a interface visual do sistema, exibindo as páginas e componentes reativos, consumidos os serviços disponibilizados pela API REST do Back-end, e executando as validações de formulários coleados ao lado de cliente-servidor (client-server). Por não possuir conectividade direta ao banco de dados, toda a comunicação com a camada lógica de negócio ocorre por requisições HTTP/REST sobre protocolo de comunicação HTTPS, utilizando a biblioteca Axios como cliente HTTP. A escolha do React JavaScript como framework principal permite a construção de uma interface responsiva corresponde a compatibilidade presente em requisitos: "RNF-PAN-021 – Responsividade", "RNF-PAN-025 – Usabilidade de Interface" e "RNF-PAN-028 – Compatibilidade entre Navegadores".
- Camada de Aplicação (Backend): Seção que constitui o núcleo de processamento do sistema, residindo todas as regras de negócio, os serviços de domínio e os mecanismos de integração com serviços externos. Esta camada gerencia os fluxos de cadastro e autenticação de usuários, catálogo e busca de produtos, carrinho de compras, pedidos; pagamentos, relatórios e administração. O acesso à camada de persistência é realizado por meio da integração dos recursos do Spring Data JPA (Java Persistence API) com Hibernate, e o controle de autenticação e autorização é reforçado pelo Spring Security em conjunto com o JWT (JSON Web Token) e OAuth 2.0. Devido com o acesso irrestrito às demais camadas, é direcionada a realização de regras que determinam quais são as operações permitidas em cada contexto existente ao sistema, atendendo a funcionalidade emitida ao requisito "RF-PAN-03 – Controle de Acesso".
- Camada de Persistência: Esta camada é responsável pelo armazenamento permanente, garantindo a integridade referencial e execução de consultas de transações sobre os dados do sistema. O banco de dados MySQL armazena todas as entidades de domínio: usuários, produtos, pedidos, itens de pedido, pagamentos, avaliações, notificações e histórico de transações. O acesso é restrito exclusivamente à Camada de Aplicação, por meio da conexão JDBC, gerenciada pela Hibernate JPA. O Front-end e o Serviços Externos não possuem acesso direto à camada de persistência em nenhuma circunstância, garantindo a integridade e segurança de dados em conformidade com os requisitos: "RNF-PAN-023 - Autenticação Segura" e "RNF-PAN-035 - Proteção de Dados".
- Serviços Externos: Os serviços externos compõem a camada de integrações com a plataforma de serviços terciários. A camada é constituída por dois dispositivos externos: o "Google Login API" responsável pela autenticação de usuários via protocolo OAuth 2.0 ("RF-PAN-014 - Integração com Redes Sociais") e a "PagBank API", responsável pelo processamento de transações financeiras ("RF-PAN-013 - Método de Pagamento"). Ambos os serviços externos não possuem acesso direto à camada de persistência, pois os dados

--	--

retornados são recebidos e persistidos pela camada lógica, que mantém o controle total sobre o que é armazenado.

9.2. Fronteiras de Camadas e Fluxos de Comunicações

As fronteiras arquiteturais do sistema definem as regras de comunicação entre camadas e os mecanismos de transporte utilizados em cada interação. O fluxo de comunicação segue a seguinte sequência:

1. O usuário acessa o sistema por meio do navegador web via HTTPS.
2. O Front-end React, hospedado no Servidor VPS, renderiza a interface e realiza as requisições ao Back-end através de chamadas REST API sobre HTTPS.
3. O Back-end Spring Boot recebe as requisições, processando as regras de negócios e de persistências através da conexão JDBC com o Banco de Dados MySQL.
4. Quando necessário, o Back-end aciona os Serviços Externos: a Google Login API para validação de credenciais e emissão de tokens OAuth 2.0, e a PagBank API para processamento de transações de pagamento (em comunicações via HTTPS).

Contudo, nenhuma camada aplicada viola as fronteiras definidas, impossibilitando alguma interferência entre a camada visual, camada lógica, camada de persistência e os serviços externos. Assegurando o isolamento, a segurança e a manutenibilidade de cada processo e estrutura presente no sistema.

9.3. Lista de Ferramentas por Camadas

A seguir, são listadas todas as ferramentas, tecnologias e serviços utilizados em cada camada presente na arquitetura implementada ao sistema.

Camada de Apresentação (Front-end)	
Ferramenta	Descrição
Axios	Biblioteca HTTP para JavaScript, utilizada no consumo de API REST do Back-end a partir do Front-end.
CSS3	Linguagem de estilização responsável pela apresentação visual, responsividade e adaptação do layout a diferentes dispositivos e resoluções.
HTML5	Linguagem de marcação para estruturação semântica das páginas web da aplicação.
JavaScript ES6+	Linguagem de programação base do Front-end, utilizada na camada lógica de interação, manipulação de estado e consumo de API.
React JavaScript	Biblioteca JavaScript para construção de interfaces de usuário reativas e baseadas em componentes. Responsável pela renderização das páginas e elementos visuais do sistema.

--	--

--	--

Tabela 9.3.1

Camada Lógica (Back-end)

Ferramentas	Descrições
Hibernate	Implementação JPA utilizada para mapeamento objeto-relacional (ORM), traduzindo as entidades Java em registros no banco de dados MySQL.
Java 21	Linguagem de programação principal do Back-end com configuração simplificada. Responsável pela estruturação da API REST e pela inicialização da aplicação.
JWT (Json Web Token)	Mecanismo de autenticação baseado em tokens assinados, utilizado para manter sessões autenticadas de forma stateless entre Front-end e Back-end.
Maven	Ferramenta de gerenciamento de dependências e automação de build no projeto back-end, responsável pela compilação, empacotamento e execução da aplicação.
OAuth 2.0	Protocolo de autorização utilizado na integração com o Google Login API para autenticação de usuários via conta Google (RF-PAN-014).
Spring Boot	Framework Java para a construção de aplicações back-end com configuração simplificada. Responsável pela estruturação da API REST e pela inicialização da aplicação.
Spring Data JPA	Módulo do Spring Framework para abstração de acesso a dados por meio da Java Persistence API, reduzindo o código boilerplate nas operações de persistências.
Spring MVC	Módulo do Spring Framework para implementação do padrão "Model-View-Controller", estruturando o mapeamento de rotas e controllers da API.
Spring Security	Módulo do Spring Framework para controlar de autenticação, autorização e proteção de endpoints. Gerencia o acesso por perfil de usuário (RF-PAN-03).

Tabela 9.3.2

Camada de Persistência (Banco de Dados)

Ferramentas	Descrições
Hibernate (JPA) Java Persistence API	Framework de mapeamento de objeto-relacional que abstrai as operações JDBC, permitindo trabalhar com entidades Java ao invés de SQL diretamente.
JDBC Java Database Connection	Interface de baixo nível para conexão e execução de operações diretamente no banco de dados MySQL a partir da aplicação Java.
MySQL	Sistema de Gerenciamento de Banco de Dados Relacional (SGBD) utilizado para armazenamento persistente de dados no sistema.
Spring Data JPA	Camada de abstração sobre o Hibernate que simplifica consultas e operações CRUD por meio de repositórios declarativos.

--	--

Tabela 9.3.3

Infraestrutura e Hospedagem

Ferramentas	Descrições
Docker	Plataforma de containerização utilizada para empacotar e executar os componentes da aplicação em ambientes isolados e reproduzíveis, facilitando o deploy e a escalabilidade
Ngix/Apache	Servidor web e proxy reverso por receber as requisições HTTPS externas e direcioná-las aos serviços internos do servidor VPS.
VPS Linux	Servidor Virtual Privado com sistema operacional em Linux, responsável por hospedar e executar todos os componentes internos do sistema (Front-end/Back-end/Banco de Dados).

Tabela 9.3.4

Serviço Externos

Ferramentas	Descrições
Google Login API (OAuth 2.0)	Serviço externo de autenticação fornecido pelo Google. Utilizado para validar credenciais de usuário, emitir de tokens OAuth 2.0 e gerenciar sessões autenticadas via conta Google.
PagBank API	Gateway de pagamento integrado ao sistema para processamento de transações financeiras, aceitando métodos como Cartão de Crédito/Débito, Boleto Bancário e Pix.

Tabela 9.3.5

10. Visão de Dados

Esta seção descreve a perspectiva de armazenamento de dados persistentes do Sistema E-commerce Panini (S.E.P), apresentando as principais entidades de domínio, seus relacionamentos e as estratégias de persistência adotadas. O sistema utiliza o MySQL como Sistema Gerenciador de Banco de Dados Relacional (SGBD), hospedado no Servidor VPS Linux.

O acesso ao banco de dados é realizado exclusivamente pela Camada de Aplicação (Back-end Spring Boot), por meio do Spring Data JPA com Hibernate como implementação ORM, e conectividade via JDBC. Nenhuma outra camada possui acesso direto à camada de persistência. As entidades persistentes do sistema são derivadas dos pacotes de domínio definidos na Visão Lógica (Seção 6.0) e estão organizadas conforme a tabela a seguir:

Entidade	Pacote de Origem	Principais Atributos Persistentes
Cliente	dominio.usuario	id_cliente, nm_cliente, nm_email_cliente, nm_logradouro, ds_complemento,

--	--

Administrador	dominio.usuario	id_admin, nm_admin, nm_email_admin, lc_nivel_acesso_usuario
Notificacao	dominio.usuario	id_notificacao, ds_mensagem, dt_emissao_pedido, cd_envio_destinatario
Produto	dominio.catalogo	id_produto, nm_produto, vl_preco_produto, qt_estoque_produto
Catalogo	dominio.catalogo	nm_produto, vl_preco_catalogo, qt_produto_catalogo
Pedido	dominio.pedido	id_pedido, ic_status_pedido, dt_emissao_pedido, vl_subtotal_pedido
ItemPedido	dominio.pedido	id_item_pedido, qt_item_pedido, vl_preco_unitario_item_pedido, vl_subtotal_item_pedido
Historico	dominio.pedido	id_pedido, ic_status_pedido, dt_emissao_pedido, vl_subtotal_pedido
Pagamento	dominio.pagamento	id_pagamento, dt_emissao_pagamento, vl_pagamento, ic_status_pagamento
NotaFiscal	dominio.pagamento	id_pagamento, dt_emissao_pagamento, vl_pagamento, ic_status_pagamento
Venda	dominio.vendas	id_venda, dt_emissao_venda, vl_total_venda, ic_status_venda
ItemVenda	dominio.vendas	Id_item_venda, qt_item_venda, vl_preco_unitario_item_venda, vl_subtotal_item_venda

Tabela 10.1

A estratégia de persistência adota o padrão Repository, implementado por componentes *ClienteRepository*, *ProdutoRepository*, *PedidoRepository* e *PagamentoRepository* presentes na camada de persistência (Seção 6.1). Cada repositório encapsula as operações de criação, leitura, atualização e remoção (CRUD) sobre seu respectivo conjunto de entidades, promovendo o desacoplamento entre as regras de negócio e os mecanismos de armazenamento.

Em conformidade com o padrão arquitetural CQRS implantado, as operações de escrita (commands – cadastros, atualizações, remoções) e de leitura (queries – consultas, relatórios, listagens) trafegam por

--	--

subsistemas distintos dentro da Camada de Aplicação, contribuindo para a rastreabilidade e escalabilidade das operações sobre os dados persistidos.

--	--

11. Referências

FOWLER, Martin. **CQRS**. Martin Fowler, 14 jul. 2011. Disponível em: <https://martinfowler.com/bliki/CQRS.html>. Acesso em: 05 jun. 2026.

KRUCHTEN, Philippe. **Architectural Blueprints: The “4+1” View Model os Software Architecture**. Disponível em: <https://www3.software.ibm.com/ibmdl/pub/software/rational/web/whitepapers/2003/Pbk4p1.pdf>. Acesso em: 05 jun. 2026.

MICROSOFT. **Padrão CQRS (Segregação de Responsabilidade de Consulta de Comando)**. Microsoft Learn. Disponível em: <https://learn.microsoft.com/pt-br/azure/architecture/patterns/cqrs>. Acesso em: 05 jun. 2026.

OMG – OBJECT MANAGEMENT GROUP. **Unified Modeling Language (UML)**. Disponível em: <https://www.omg.org/spec/UML/>. Acesso em: 05 jun. 2026.

SOMMERVILLE, Ian. **Engenharia de Software**. 9. ed. São Paulo: Person Prentice Hall, 2011. Disponível em: <https://www.facom.ufu.br/~william/Disciplinas%202018-2/BSI-GSI030-EngenhariaSoftware/Livro/engenhariaSoftwareSommerville.pdf>. Acesso em: 05 jun. 2026.

WIKIPEDIA. **Engenharia de Software Orientada a Modelos**. Disponível em: https://pt.wikipedia.org/wiki/Engenharia_de_software_orientada_a_modelos. Acesso em: 05 jun. 2026.